



BREEZY
Développement
d'application distribuées

CESI —
ÉCOLE D'INGÉNIEURS

Projet Développement d'Application Distribuées

Réalisé par

Alexandre RIVET

Philippe LUU

Romain TOUZE

Maxime STOFFEL

Projet encadré par

Mathieu DESHORS / Ali SKAF / Djamel AOUAM

Année universitaire 2026

Livrable - 25/06/2026

Informations générales

Versions du document :

Ce tableau retrace toutes les versions du document :

Version	Date	Auteur(s)	Objet
1.0	01/06/2026	LUU Philippe, RIVET Alexandre, TOUZE Romain, STOFFEL Maxime	Création du document
1.1	23/06/2026	LUU Philippe, RIVET Alexandre	Revue du document et des parties : 3 et 4

Liste de diffusion :

Ce tableau répertorie tous les destinataires ayant accès au document :

Entreprise / Organisme	Destinataire(s)	Contacts	Rôle
Breezy	LUU Philippe	philippe.luu@breezy.fr	Chef de projet & Scrum Master
Breezy	RIVET Alexandre	alexandre.rivet@breezy.fr	Architecte / Lead Dev
Breezy	TOUZE Romain	romain.touze@breezy.fr	Ingénieur d'études et développement
Breezy	STOFFEL Maxime	maxime.stoffel@breezy.fr	Ingénieur d'études et développement
CESI	AOUAM Djamel	daouam@cesi.fr	Directeur informatique
CESI	SKAF Ali	askaf@cesi.fr	Responsable informatique
CESI	DESHORS Mathieu	mdeshors@cesi.fr	Responsable informatique

Synthèse Managériale

Ce document présente le bilan de Breezy, un réseau social léger inspiré de Twitter/X, conçu et réalisé par une équipe de quatre élèves ingénieurs FISA INFO A3 dans le cadre du module Développement d'applications distribuées. L'objectif : offrir une application web et mobile permettant de publier des messages courts (280 caractères), de suivre d'autres membres, d'échanger et de réagir, avec une expérience simple, rapide et fluide. Ce résumé s'adresse à un lecteur décideur ; les éléments techniques sont développés dans le corps du rapport.

L'application est complète et pleinement fonctionnelle. Elle couvre l'intégralité des onze fonctionnalités obligatoires du cahier des charges (création de compte, connexion sécurisée, publication, fil d'actualité, "j'aime", commentaires, réponses, abonnements et profils) ainsi que l'intégralité des douze fonctionnalités optionnelles (mots-clés et recherche, notifications en temps réel, messagerie privée, ajout d'images et de vidéos, signalement et modération, interface multilingue et thèmes personnalisables). Plusieurs fonctions vont même au-delà de la demande initiale : une messagerie privée dont les messages sont chiffrés (illisibles, y compris par la plateforme), des sondages, des citations, des favoris classés en collections, des comptes privés sur demande, le support de douze langues (dont l'arabe, écrit de droite à gauche) et un espace d'administration permettant de superviser l'état du service. Les quatre profils prévus: Visiteur, Utilisateur, Modérateur et Administrateur sont opérationnels.

Une base solide et sécurisée. Breezy s'appuie sur une architecture moderne et modulaire : chaque grande fonction (comptes, publications, messagerie, médias, notifications, modération...) est prise en charge par un composant indépendant. Ce découpage renforce la fiabilité (une fonction peut évoluer sans perturber les autres), la sécurité et la capacité à grandir avec le nombre d'utilisateurs. La protection des accès et des données personnelles a été traitée comme une priorité : connexion sécurisée, gestion des sessions et droits différenciés selon le profil. L'application est par ailleurs conçue pour être déployée de façon standardisée et reproductible, ce qui la rend simple à exploiter et à faire évoluer. Les choix techniques détaillés figurent aux sections 4 et 5 du rapport.

Le projet a été mené sur le mois de juin 2026 selon une démarche agile et itérative, avec un suivi rigoureux des tâches sur un outil de gestion de projet et une revue systématique du

travail entre membres de l'équipe. Près de 200 tâches ont été traitées, organisées en six phases : conception et mise en place, fonctionnalités obligatoires, fonctionnalités optionnelles, fonctionnalités additionnelles, corrections et stabilisation, puis tests et qualité. La qualité a été contrôlée par une large batterie de tests automatisés couvrant plus de 95 % du code sur l'interface et plusieurs composants. Les quelques retards (mise en service, temps de première connexion, fiabilisation de la messagerie) ont été identifiés et résorbés en fin de projet.

Roadmap du projet Breezy — 6 phases

Période d'activité principale par phase (juin 2026) · barre = p10-p90 des clôtures · ♦ médiane · n = nombre d'issues

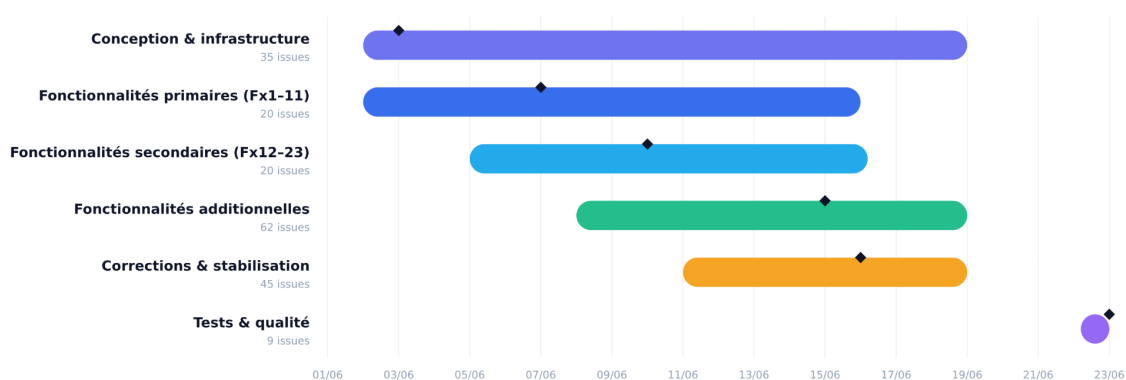


Figure 1 : Roadmap du projet Breezy en six phases

Le blanc visible sur le dernier trait correspond simplement au week-end. Le graphique est basé sur des dates calendaires : lorsqu'une phase commence un vendredi et se termine le lundi, le samedi et le dimanche sont inclus dans l'intervalle affiché, même s'il n'y a pas eu d'activité pendant ces jours-là. Ce blanc permet donc de distinguer les jours non travaillés des jours réellement actifs.

Breezy a ses débuts devant survivre sur les investissements acquis lors des levées de fond, cela permet un accès complet à l'application sans publicités ni restrictions, le but sera d'attirer un maximum d'utilisateurs jusqu'à un certain seuil et un temps donné afin de commencer à mettre la seconde phase de la stratégie économique.

Breezy sera pensé pour être financé par la publicité : des posts sponsorisés intégrés au fil, vendus au CPM (coût pour mille impressions) et au CPC (coût par clic), complétés par des tendances sponsorisées et trois formules annonceurs (Starter 99 €/mois, Pro 490 €/mois,

Business 1 900 €/mois). Le coût de R&D du projet réalisé est valorisé à environ 28 000 €. Sur la base d'hypothèses prudentes de croissance (50 000 utilisateurs actifs mensuels la première année, 600 000 la troisième), la projection à trois ans fait apparaître un seuil de rentabilité cumulé atteint au cours de la deuxième année ($\approx 14^{\text{e}}$ mois), confirmant la viabilité économique du modèle. Par la suite, un abonnement optionnel **Breezy+** sera disponible à 4,99 €/mois (expérience sans publicité, badge, fonctions étendues) qui va constituer un revenu secondaire.

En conclusion, Breezy constitue un produit cohérent, sécurisé et documenté, qui répond aux exigences du cahier des charges, démontre la maîtrise d'une architecture moderne et modulaire et propose un modèle de financement crédible. Les limites identifiées (renforcement de la sécurité, tests complémentaires, supervision) sont explicitées et priorisées comme axes d'évolution.

Indicateur clé	Valeur(s)
Fonctionnalités primaires	11/11
Fonctionnalités secondaires	12/12
Rôles fonctionnels	4/4
Composants applicatifs indépendants	11
Solutions de stockage de données	3
Langues d'interface	12
Tâches traitées	210 + 6 phases
Couverture de tests	> 95 %
Coût de R&D valorisé	~ 28 000 €
Seuil de rentabilité	~ 14 mois

Tableau 1 : Indicateurs clés du projet

Sommaire

Informations générales.....	3
Versions du document :	3
Liste de diffusion :	3
Synthèse Managériale.....	4
Sommaire.....	7
1. Introduction.....	9
2. Objectifs et attentes du projet.....	10
3. Analyse du cahier des charges et priorisation.....	12
3.1 Analyse du cahier des charges.....	12
3.2 Couverture fonctionnelle du projet.....	13
4. Architecture générale de l'application.....	16
4.1 Adaptation du schéma cible.....	18
4.2 Rôle de chaque service.....	18
5. Stack technique et justification des choix.....	22
5.1 Stack technique.....	22
5.2 Justifications sur les choix techniques.....	23
6. Méthodologie de travail, ressources et étapes de réalisation.....	25
6.1 Méthodologie et organisation.....	25
6.2 Outils de pilotage et de collaboration.....	26
6.3 Planning, phases et jalons.....	27
6.4 Répartition des rôles et responsabilités.....	28
6.5 Gestion des risques et des retards.....	30
6.6 Ressources mobilisées.....	31
7. Description détaillée des fonctionnalités principales.....	32
7.1 Authentification, comptes et sécurité.....	32
7.2 Publications et fil d'actualité.....	34
7.3 Profils et relations sociales.....	35
7.4 Explorer, recherche, hashtags et tendances.....	36
7.5 Notifications temps réel.....	36
7.6 Messagerie privée chiffrée.....	37
7.7 Médias et stockage objet.....	39
7.8 Paramètres, thème et internationalisation.....	39
7.9 Modération et administration.....	40
7.10 Mode visiteur.....	41
7.11 Matrice des rôles.....	42
8. Interfaces utilisateur, UX et Maquettes.....	44
Maquette 1 - Écran d'authentification.....	44
Maquette 2 - Fil d'actualité desktop.....	46

Maquette 3 - Fil d'actualité mobile.....	48
Maquette 4 - Page profil.....	51
Maquette 5 - Modale de publication.....	53
Maquette 6 - Messagerie.....	55
Maquette 7 - Paramètres.....	57
Maquette 8 - Modération et administration.....	60
9. Tests, qualité, déploiement et documentation.....	64
10. Sécurité, audits et tests d'intrusion.....	67
10.1 Sécurité dès la conception.....	67
10.2 Protection des comptes et des données.....	68
10.3 Test d'intrusion (pentest).....	69
9. Difficultés rencontrées et limites.....	71
10. Axes d'amélioration et évolutions possibles.....	73
12.1 Axes d'améliorations.....	73
12.2 Modèle économique et perspectives de financement.....	74
11. Conclusion.....	76
12. Table des figures.....	77

1. Introduction

Breezy est un projet d'élèves ingénieurs de réseau social léger, inspiré du fonctionnement général de Twitter/X. L'application permet à un utilisateur de créer un compte, de publier des messages courts, de suivre d'autres profils, d'interagir avec des publications, d'envoyer des messages privés et de recevoir des notifications. L'objectif n'était pas de reproduire toutes les capacités d'une plateforme industrielle, mais de concevoir une application cohérente, utilisable et techniquement réaliste à notre niveau d'élèves ingénieurs de première année du cycle FISA INFO A3.

Le projet a été pensé autour d'une architecture distribuée. Le frontend est développé avec Next.js 14, React 18 et TypeScript, tandis que le back-end est découpé en microservices indépendants écrits en Go avec Gin. Cette séparation a permis de répartir clairement les responsabilités entre l'authentification, les utilisateurs, les profils, les publications, les messages, les notifications, les médias et les e-mails. Docker Compose orchestre les services et leurs dépendances de données, ce qui simplifie le lancement de l'application et rend l'environnement reproductible.

Une adaptation importante a été faite par rapport au sujet initial : le cahier des charges évoquait un back-end Node.js/Express, mais l'équipe a choisi Go/Gin pour construire des services plus autonomes, compilés, performants et simples à conteneuriser. L'esprit du sujet a cependant été conservé : API RESTful sécurisée, frontend React, Docker, bases de données séparées et architecture microservices. Le rapport ne prétend donc pas que Node.js/Express a été utilisé ; il explique au contraire pourquoi ce choix a été adapté, et en assume la responsabilité technique.

Ce rapport présente successivement les objectifs du projet, l'analyse du cahier des charges, l'architecture choisie, la stack technique, la gestion de projet et le planning, les principales fonctionnalités implémentées, les maquettes d'interface, le modèle économique envisagé, la démarche qualité, puis les limites connues et les axes d'amélioration. Le ton adopté reste volontairement sobre et honnête : Breezy est un projet étudiant fonctionnel et structuré, mais il conserve des axes d'amélioration importants, notamment sur la sécurité, les tests d'intégration et l'observabilité.

2. Objectifs et attentes du projet

Le premier objectif était de proposer une application web sociale complète autour de messages courts de 280 caractères. L'utilisateur devait pouvoir s'inscrire, se connecter, publier, consulter un fil d'actualité, aimer, commenter, répondre, suivre d'autres comptes, consulter des profils et naviguer via des hashtags. L'interface devait rester simple, rapide à comprendre et utilisable sur ordinateur comme sur mobile.

Le deuxième objectif concerne l'architecture. Le cahier des charges demandait une API RESTful sécurisée, un frontend moderne, la conteneurisation avec Docker et une séparation claire des responsabilités. Nous avons conservé ces attentes, mais nous avons adapté le choix du back-end : au lieu d'utiliser Node.js/Express, nous avons développé des microservices Go/Gin. Ce choix a été fait pour obtenir des services autonomes, fortement typés, faciles à lancer indépendamment et adaptés à une architecture distribuée.

Le troisième objectif était de dépasser les fonctionnalités minimales en ajoutant plusieurs éléments proches d'une application réelle : messages privés chiffrés, notifications temps réel, médias via stockage objet, internationalisation, paramètres de confidentialité, centre de modération et page d'administration. Ces ajouts ont été développés progressivement, après la stabilisation des éléments essentiels que sont l'authentification, les profils et les publications.

Axe	Attente	Traduction en tâches
Fonctionnel	Réseau social léger	Publier, consulter un fil, suivre des comptes et interagir avec les contenus.
Technique	Architecture distribuée	Séparer le frontend, la gateway, les services métier et les bases de données.
Sécurité	Accès authentifié	JWT, refresh token, bcrypt, vérification e-mail, OAuth Google et Github, rôles et routes protégées.

UX	Interface responsive	Navigation desktop proche de X/Twitter, approche mobile-first, thème clair/sombre et accents personnalisables.
Qualité	Projet maintenable	Docker Compose, Makefile, OpenAPI/Swagger, tests Go, Vitest, CI et logs structurés.

Tableau 2 : Objectifs, attentes du projet et traduction en livrables

Ce tableau résume les attentes principales. Il montre que le projet ne se limite pas à une interface de publication : l'enjeu était aussi de structurer une application distribuée, de sécuriser les échanges, de préparer un environnement de développement exploitable par une équipe et d'envisager un modèle de financement crédible.

3. Analyse du cahier des charges et priorisation

3.1 Analyse du cahier des charges

L'analyse du cahier des charges nous a conduits à distinguer les fonctionnalités indispensables des fonctionnalités avancées. Les éléments indispensables étaient l'inscription, la connexion, la gestion des utilisateurs, les profils et les posts. Sans ces fondations, les autres modules comme les notifications, les messages privés ou la modération n'auraient pas eu de base exploitable.

Nous avons donc adopté une hiérarchisation en plusieurs niveaux. Le socle technique et fonctionnel a été traité en premier : structure des microservices, Docker Compose, bases de données, API Gateway, authentification et premier écran frontend. Ensuite, nous avons ajouté le domaine social avec les posts, likes, commentaires, abonnements et profils. Les modules plus complexes, notamment les WebSockets, les médias, la messagerie chiffrée, l'internationalisation et l'administration, ont été intégrés après stabilisation du parcours principal.

Niveau	Famille	Tâches principales
Priorité 1	Socle	Docker Compose, API Gateway, services Go, bases PostgreSQL/MongoDB/MinIO, frontend Next.js .
Priorité 2	Compte & identité	Inscription, login, JWT, refresh token, utilisateurs, profils, vérification e-mail.

Priorité 3	Réseau social	Posts, fil d'actualité, commentaires, likes, follows, hashtags, médias.
Priorité 4	Fonctions avancées	Notifications, messages E2EE, paramètres, i18n, modération, administration.
Priorité 5	Consolidation	Documentation OpenAPI finalisée, rapport, recensement des limites et axes d'amélioration

Tableau 3 : Niveaux de priorisation des tâches

Axe transversal : Qualité & industrialisation. Contrairement aux quatre niveaux ci-dessus, la qualité n'est pas une étape : elle s'applique tout au long du projet. Les conventions (Gitflow, Conventional Commits), la conteneurisation, les logs structurés (slog) et la chaîne CI (ci-go, ci-frontend, ci-integration, validation Swagger) ont été mises en place dès le socle et maintenues à chaque incrément. Seule la campagne de tests d'intégration sur bases réelles est restée partielle (cf. chapitre 10).

Cette priorisation a évité de commencer par des fonctionnalités attractives mais dépendantes d'un socle non finalisé. Par exemple, les notifications de follow ou de commentaire n'ont de sens que lorsque les utilisateurs, les abonnements et les posts sont déjà fonctionnels.

3.2 Couverture fonctionnelle du projet

Pour rendre la couverture plus lisible, nous avons rapproché les fonctionnalités attendues du cahier des charges et les modules effectivement réalisés. Certaines fonctionnalités dépassent le minimum demandé, comme les sondages, les citations, les signets en collections, la messagerie chiffrée, les profils privés, le statut de présence,

l'internationalisation et l'administration.

Fx	Fonctionnalité	Type	Etat	Mise en oeuvre
Fx1	Création de compte + validation	Primaire	Fait	auth-service + mail-service
Fx2	Authentification sécurisée	Primaire	Fait	auth-service + gateway + BFF Next.js
Fx3	Posts courts de 280 caractères	Primaire	Fait	post-service
Fx4	Messages visibles sur profil	Primaire	Fait	post-service + profil-service
Fx5	Fil des suivis / abonnements	Primaire	Fait	post-service + user-service
Fx6	Like de post	Primaire	Fait	post-service + notification-service
Fx7	Commentaire de post	Primaire	Fait	post-service
Fx8	Réponse à un commentaire	Primaire	Fait	post-service, profondeur limitée à deux niveaux
Fx9	Follow / unfollow	Secondaire	Fait	user-service, demandes pour profils privés
Fx10	Profil utilisateur	Secondaire	Fait	profil-service + media-service
Fx11	Liste des posts sur profil	Secondaire	Fait	post-service
Fx12	Hashtags	Secondaire	Fait	extraction côté serveur + liens cliquables
Fx13	Recherche par tags	Secondaire	Fait	Explorer + post-service

Livrable - 25/06/2026

Fx14	Notifications de mention	Secondaire	Fait	notification-service + WebSocket
Fx15	Notifications de likes	Secondaire	Fait	notification-service + WebSocket
Fx16	Notifications nouveaux followers	Secondaire	Fait	notification-service + événements internes
Fx17	Messages privés	Secondaire	Fait	message-service, E2EE côté client
Fx18	Images	Secondaire	Fait	media-service + MinIO
Fx19	Vidéos	Secondaire	Fait	media-service + streaming via gateway
Fx20	Signalement direct de contenu	Secondaire	Fait	report-service : signalement de posts/messages/comptes, tickets de modération, avertissements et auto-masquage
Fx21	Suspension / bannissement	Secondaire	Fait	modération/admin, désactivation ou réactivation de compte
Fx22	Interface multilingue	Secondaire	Fait	frontend, 12 langues, RTL pour arabe
Fx23	Thème personnalisé	Secondaire	Fait	next-themes, clair/sombre, accent personnalisé

Tableau 4 : Couverture fonctionnelle (Fx1 - Fx23)

Le signalement direct (Fx20) est pleinement implémenté par un service dédié, report-service : depuis l'interface, un utilisateur peut signaler un post, un message privé ou de groupe, ou un compte, en choisissant un motif (contenu inapproprié, langage offensant, spam, bug ou autre). Chaque signalement alimente un ticket de modération consultable par les modérateurs (file de tickets, réponses, changement de statut, transfert, validation, retrait de

contenu), et un post dépassant un seuil de signalements configurable est auto-masqué. Le système gère également des avertissements que l'utilisateur doit accuser réception, et reste compatible avec le chiffrement de bout en bout (l'utilisateur divulgue volontairement le message chiffré ciblé au moment du signalement).

4. Architecture générale de l'application

Breezy est organisé en quatre couches : la couche client, la couche web, la couche services et la couche données. La couche client représente les rôles User, Moderator et Administrator. La couche web correspond au frontend Next.js. La couche services contient l'API Gateway et les microservices métier. Enfin, la couche données regroupe PostgreSQL, MongoDB et MinIO.

L'API Gateway constitue le point d'entrée unique pour le frontend. Elle route les requêtes vers les services concernés, applique la configuration CORS, relaie les WebSockets et assure le streaming des médias. Les échanges principaux utilisent HTTP REST. Les fonctionnalités temps réel, comme les messages, les notifications et le bandeau de nouveaux posts, reposent sur WebSocket.

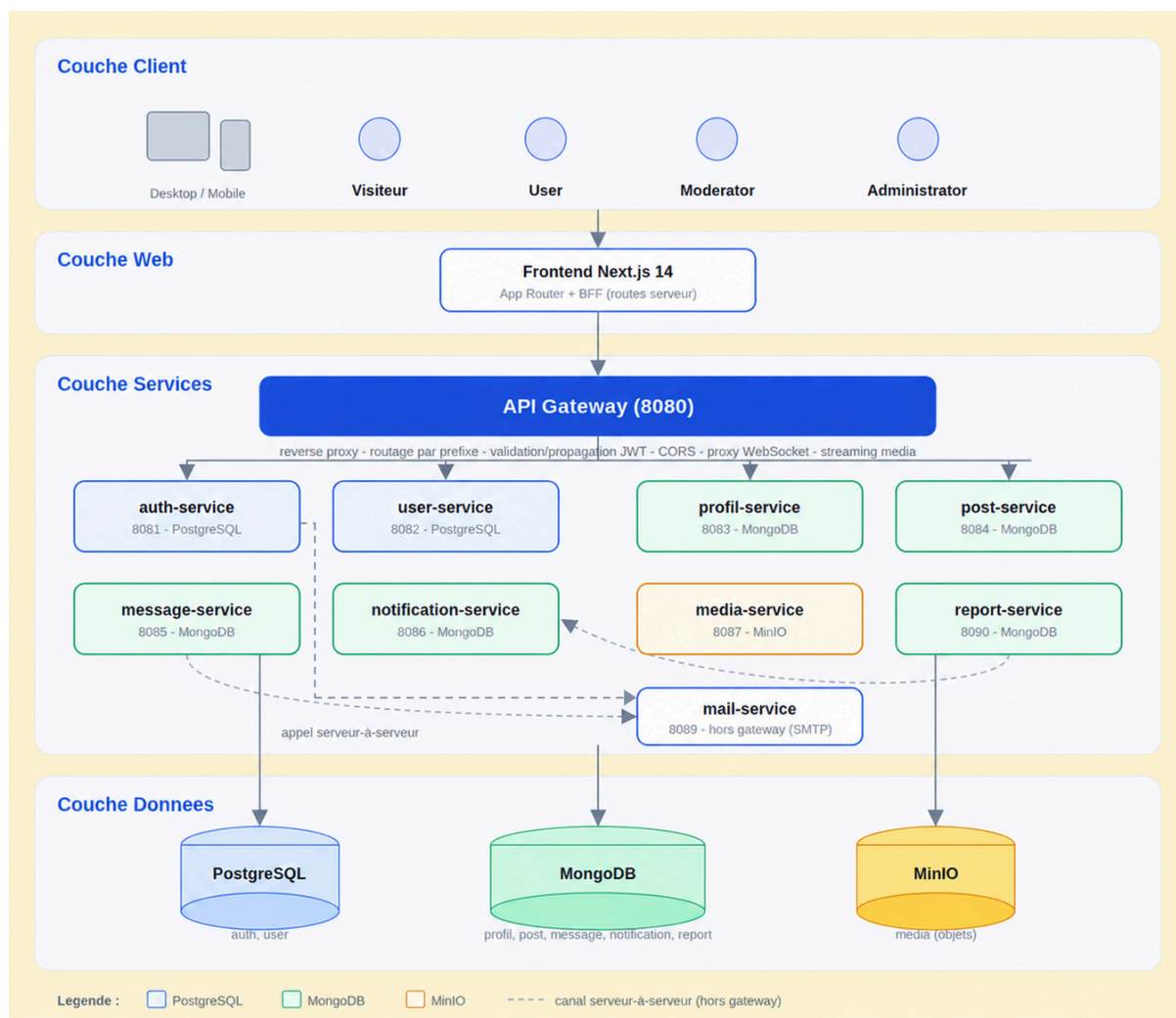


Figure 2 : Architecture cible de Breezy en quatre couches : frontend, API Gateway et neuf services métier

Le schéma montre également deux flux particuliers. Le mail-service n'est pas exposé comme une API publique : il est appelé par auth-service pour envoyer les e-mails de vérification, de réinitialisation de mot de passe et de changement d'adresse. De son côté, notification-service reçoit des événements internes émis par user-service, post-service ou message-service à l'aide d'un secret partagé. Cette exception est volontaire, car il s'agit d'événements serveur à serveur et non de requêtes utilisateur directes.

4.1 Adaptation du schéma cible

Le schéma initial du sujet mettait surtout en avant quatre services métier : authentification, utilisateurs, profils et posts. Le projet final conserve ces services, mais les complète par des services spécialisés pour les messages, notifications, médias et e-mails. Cette évolution suit un principe simple : dès qu'un domaine possède ses propres données et ses propres règles métier, il devient plus clair de l'isoler dans un service dédié.

message-service a été séparé car la messagerie possède ses propres conversations, clés, reçus et WebSockets. notification-service agrège des événements provenant de plusieurs domaines. report-service isole le signalement de contenu et la modération (tickets, avertissements, seuils d'auto-masquage), avec sa propre base. media-service reste agnostique du contenu et stocke des fichiers sous forme d'objets dans MinIO. mail-service isole la configuration SMTP et les templates d'e-mails transactionnels. Ce découpage évite de surcharger post-service ou auth-service avec des responsabilités qui ne leur appartiennent pas directement.

4.2 Rôle de chaque service

Service	Port	Données	Responsabilité
api-gateway	8080	–	Point d'entrée unique : reverse proxy, routage par préfixe, CORS, proxy WebSocket, streaming média et monitoring admin.
auth-service	8081	PostgreSQL	Inscription, connexion, bcrypt, JWT HS256, refresh tokens opaques hachés, OAuth Google, vérification e-mail, reset et comptes admin.
user-service	8082	PostgreSQL	Identité, username, rôle, statut actif, graphe social, follows, demandes de follow et préférences de langue.
profil-service	8083	MongoDB	Nom affiché, bio, avatar, bannière, localisation, site web, nationalité, genre, date de naissance, visibilité du profil.

post-service	8084	MongoDB	Posts, commentaires, réponses à deux niveaux, likes, reposts, citations, pins, sondages, hashtags, signets et filtres de visibilité.
message-service	8085	MongoDB	Conversations, DM, groupes, communautés, messages chiffrés, WebSocket, reçus, typing et pièces jointes chiffrées.
notification-service	8086	MongoDB	Notifications agrégées, ingestion d'événements internes, badge utilisateur et WebSocket.
report-service	8090	MongoDB	Signalement de posts, messages et comptes, motifs, tickets de modération, réponses, avertissements et auto-masquage.
media-service	8087	MinIO	Stockage objet des avatars, bannières, médias de posts, vidéos, blobs et pièces jointes chiffrées.
mail-service	8089	SMTP console /	Envoi d'e-mails de vérification, reset, changement d'e-mail et mot de passe temporaire pour compte créé par admin.

Tableau 5 : Rôle de chaque service

La séparation par service impose une règle de propriété des données : une donnée métier appartient à un seul service. Par exemple, le username appartient à user-service, tandis que la bio et la visibilité du profil appartiennent à profil-service. Les vues agrégées sont reconstruites côté frontend ou via des appels contrôlés lorsque cela est nécessaire.

Le schéma ci-dessous récapitule cette règle de propriété : chaque catégorie de donnée est rattachée à un service unique et à sa base, les vues agrégées étant recomposées par le frontend ou le BFF sans transaction inter-service.

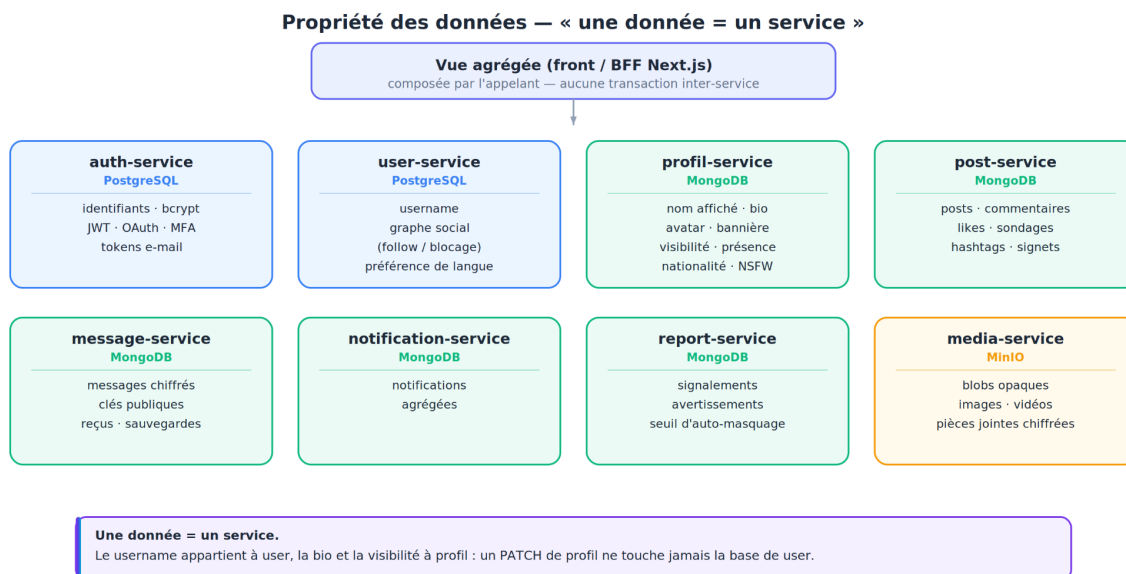


Figure 3 : Propriété des données : « une donnée = un service »

4.3 Flux techniques et règles d'architecture

Principe	Application dans Breezy
Point d'entrée unique	Le frontend appelle l'API Gateway, qui relaie les requêtes HTTP, les WebSockets et les médias. Les bases de données ne sont jamais exposées au client.
BFF Next.js	Certaines routes serveur Next.js posent le cookie refresh httpOnly, gèrent l'échange OAuth Google et masquent des clés serveur comme la traduction.
JWT et refresh	L'access token court est envoyé en Bearer ; le refresh token opaque est stocké dans un cookie httpOnly et renouvelé via une route BFF same-origin.
Événements internes	Les notifications sont alimentées par des appels serveur à serveur vers /internal/events avec un secret partagé.
Schémas autonomes	Chaque service initialise ou resynchronise son schéma au démarrage avec une logique idempotente de type EnsureSchema.

Migrations prudentes	Les champs ajoutés à MongoDB sont accompagnés de valeurs par défaut ou de backfills afin de ne pas bloquer les anciens documents.
Médias	Les vidéos et fichiers sont servis via la gateway avec support des requêtes Range ; MinIO reste caché derrière media-service.

Tableau 6 : Principes et flux d'architecture

Ces règles sont importantes parce qu'elles évitent un couplage excessif. Elles permettent aussi de justifier les choix d'architecture devant le cahier des charges : Breezy n'est pas seulement une juxtaposition de services, mais une application où chaque domaine possède une frontière claire.

5. Stack technique et justification des choix

5.1 Stack technique

Le frontend utilise Next.js 14 avec React 18 et TypeScript. Ce choix nous a donné une base moderne avec l'App Router, des composants réutilisables et une bonne intégration des routes applicatives. Tailwind CSS, shadcn/ui et Radix UI ont servi à construire rapidement une interface cohérente. Lucide-react apporte les icônes, tandis que next-themes gère les modes clair et sombre.

Le back-end est composé de microservices Go utilisant Gin. Même si le sujet initial mentionnait Node.js/Express, nous n'avons pas prétendu l'avoir utilisé. Nous avons gardé l'objectif d'une API RESTful sécurisée et avons remplacé Express par Gin pour bénéficier d'un runtime compilé, de binaires légers, d'une bonne lisibilité des handlers et d'une conteneurisation simple mais aussi car l'équipe voulait découvrir ce langage. Cette adaptation reste cohérente avec le besoin d'indépendance entre services.

Domaine	Technologies	Justification
Frontend	Next.js 14, React 18, TypeScript	Interface typée, responsive, structurée en composants et adaptée aux routes publiques/protégées.
UI	Tailwind CSS, shadcn/ui, Radix UI, lucide-react	Design mobile-first, composants accessibles, icônes homogènes et itération rapide.
Back-end	Go 1.25, Gin	Services indépendants, routes REST, middlewares JWT, logs structurés et compilation en binaire.
Données	PostgreSQL, MongoDB, MinIO	Relationnel pour auth/user, documents pour social, objets pour médias.

Sécurité	JWT, refresh token, bcrypt, OAuth Google et Github, WebSocket authentifié	Contrôle des sessions, mots de passe hachés et accès par rôle.
Industrialisation	Docker Compose, Makefile, GitHub Actions, OpenAPI/Swagger	Lancement reproductible, automatisation des tests et documentation de l'API.

Tableau 7 : Stack technique et justifications

Cette stack a été choisie pour rester réaliste dans un contexte étudiant. Elle permet de travailler séparément sur le frontend et les services, tout en conservant une orchestration commune. Les bases de données ont été choisies selon la nature des données : PostgreSQL pour les relations fortes, MongoDB pour les contenus sociaux plus flexibles, et MinIO pour les fichiers binaires.

5.2 Justifications sur les choix techniques

Next.js a été préféré à une simple SPA parce que Breezy possède plusieurs parcours publics et protégés, des redirections, un contrôle d'accès par rôle et des routes BFF. Ces routes serveur sont utiles pour poser le cookie refresh httpOnly, gérer l'échange OAuth Google, appeler certaines API côté serveur et éviter d'exposer des secrets dans le navigateur.

Go/Gin a été retenu pour le back-end car chaque service peut être compilé, testé et conteneurisé indépendamment. Chaque service possède son propre go.mod, son Dockerfile, ses fichiers de configuration, son router, ses handlers, ses modèles et sa logique métier. Les Dockerfiles utilisent une construction multi-étapes : compilation dans une image Go/Alpine, puis exécution dans une image runtime Alpine avec un utilisateur non-root.

Le choix PostgreSQL/MongoDB/MinIO répond à une logique de responsabilité. L'authentification et le graphe social nécessitent des relations fortes et des contraintes ; PostgreSQL est donc adapté. Les profils, posts, messages et notifications évoluent plus vite et contiennent de nombreux champs optionnels ; MongoDB rend ces documents plus souples. Les fichiers binaires ne sont pas stockés en base : MinIO sert de stockage objet pour éviter d'alourdir les collections.

Enfin, le client HTTP du frontend centralise l'injection du Bearer token, la gestion d'un 401 et le rafraîchissement de session. Le refresh est réalisé de manière contrôlée (single-flight) afin d'éviter plusieurs appels concurrents au même moment. Cette logique améliore l'expérience utilisateur, mais elle reste une zone sensible puisque l'access token est conservé côté navigateur.

6. Méthodologie de travail, ressources et étapes de réalisation

6.1 Méthodologie et organisation

La méthodologie suivie a été agile et incrémentale, adaptée à une équipe de quatre personnes travaillant sur une période courte et intense (mois de juin 2026). Plutôt qu'un cycle en V rigide, nous avons fonctionné par itérations successives organisées autour du flux GitHub : chaque tâche faisait l'objet d'une issue, développée sur une branche dédiée, intégrée via une Pull Request relue par un autre membre, puis fusionnée après passage de la CI. Les commits suivent la convention Conventional Commits (feat:, fix:, test:, docs:...), ce qui rend l'historique lisible et facilite la génération de notes de version.

Nous avons d'abord analysé les besoins fonctionnels et techniques, puis défini les frontières des microservices. Cette étape était déterminante : une mauvaise répartition aurait entraîné soit un couplage trop fort, soit une multiplication d'appels inutiles entre services. La deuxième étape a porté sur les modèles de données (tables PostgreSQL pour l'identité, collections MongoDB pour le social, bucket MinIO pour les fichiers). Le développement a ensuite été mené par blocs fonctionnels, en branchant progressivement les écrans du frontend sur les clients d'API au fur et à mesure de la disponibilité des routes.

Étape	Travail réalisé
Analyse	Lecture du sujet, identification du périmètre, adaptation Node/Express vers Go/Gin.
Conception	Découpage des services, choix des bases, définition des responsabilités et des flux.
Réalisation	Développement des services Go, clients frontend, composants UI et routes BFF.
Intégration	Raccordement via API Gateway, Docker Compose, WebSockets, médias et e-mails.
Validation	Tests Go, Vitest, vérifications manuelles end-to-end et documentation OpenAPI.

Tableau 8 : Etapes de réalisation

Cette méthode n'est pas une démarche "agile" (pas de sprints calendaires formels de deux semaines compte tenu de la durée), mais elle en reprend les principes : itérations courtes, intégration continue, revue par les pairs, priorisation par la valeur et amélioration continue. À chaque nouvelle fonctionnalité, nous vérifions le quadruplet parcours utilisateur → route back-end → modèle de données → intégration gateway.

6.2 Outils de pilotage et de collaboration

Le suivi s'est appuyé sur GitHub Projects comme tableau de bord central (équivalent Trello/Kanban), avec des colonnes Backlog → In progress → In review → Done et un étiquetage par domaine. Les tâches ont été regroupées par service : auth, gateway, user, profil, post, message, notification, media, frontend, tests et documentation. Cette organisation a permis de travailler en parallèle tout en gardant une vision globale du parcours utilisateur.

Outil	Usage
GitHub (dépôt mono-repo)	Code, branches, Pull Requests, revue par les pairs, historique conventionnel.
GitHub Projects	Tableau Kanban, suivi des issues, étiquettes par domaine et par phase.
GitHub Actions (CI)	Workflows Go et frontend, intégration Docker, validation Swagger, contrôle de dérive documentaire.
Docker Compose	Environnement local reproductible (services, bases, MinIO, frontend).
Makefile	Commandes normalisées : make env, make dev, make up, make down, make reset, make swagger.
OpenAPI/Swagger	Contrat d'API par service, documentation publiée et consultable.
Discord / échanges synchrones	Coordination quotidienne, levée des points bloquants, décisions rapides.

Tableau 9 : Outils de pilotage et de collaboration

La configuration est restée externalisée : un fichier `.env.example` existe à la racine et dans les services concernés, tandis que les secrets réels ne sont pas versionnés. Le `Makefile` regroupe les commandes courantes (préparation de l'environnement, lancement, arrêt, réinitialisation, génération de documentation).

6.3 Planning, phases et jalons

Le projet s'est déroulé sur le mois de juin 2026. Le suivi des issues GitHub permet de reconstituer le planning réel, organisé en six phases qui se recouvrent partiellement : l'infrastructure et les fonctionnalités primaires ont démarré ensemble, les fonctionnalités secondaires et additionnelles se sont enchaînées, puis une phase dédiée de corrections/stabilisation et une campagne finale de tests ont clôturé le projet.

Phase	Issues	Période	Jalon de sortie
Conception infrastructure &	35	01 → 19 juin	Socle conteneurisé : mono-repo, services squelettes, gateway, bases, CI, hot-reload.
Fonctionnalités primaires (Fx1–11)	20	01 → 17 juin	Parcours complet : inscription, login JWT, posts, fil, likes, commentaires, follow, profil.
Fonctionnalités secondaires (Fx12–23)	20	01 → 18 juin	Hashtags, notifications, messages privés, médias, i18n, thèmes, modération.
Fonctionnalités additionnelles	62	02 → 23 juin	Au-delà du sujet : E2EE, sondages, citations, signets, profils privés, admin.

Corrections & stabilisation	45	08 → 22 juin	Résolution des bugs (responsive, production, messagerie, traduction).
Tests & qualité	12	01 → 23 juin	Couverture > 95 % frontend et services clés, documentation finalisée.

Tableau 10 : Planning par phases et jalons

Les principaux jalons ont été : (J1) socle technique conteneurisé opérationnel, (J2) parcours utilisateur primaire de bout en bout, (J3) intégration des fonctionnalités secondaires et avancées, (J4) déploiement en production sur VPS, (J5) campagne de stabilisation et de tests. Le découpage en phases recouvrantes visible sur la [Figure 1](#) reflète une réalité de projet : on ne fige pas l'infrastructure avant de commencer les fonctionnalités, on les fait progresser ensemble en maintenant l'intégration continue.

6.4 Répartition des rôles et responsabilités

L'équipe est composée de quatre élèves ingénieurs. Les responsabilités ont été réparties par domaine de compétence, tout en maintenant une polyvalence permettant à chacun d'intervenir sur l'ensemble du projet (condition nécessaire à la soutenance, où chaque membre doit maîtriser l'intégralité du système).

Membre	Rôle projet	Contribution principale
Philippe LUU	Chef de projet & Scrum Master	Pilotage, priorisation, suivi GitHub Projects, intégration et coordination.
Alexandre RIVET	Architecte / Lead Dev	Architecture microservices, gateway, choix techniques, revue de code.
Romain TOUZE	Ingénieur d'études et développement	Développement back-end (services Go), bases de données, tests.

Maxime STOFFEL	Ingénieur d'études et développement	Développement frontend, UI/UX, internationalisation, intégration API.
----------------	-------------------------------------	---

Tableau 11 : Répartition des rôles de l'équipe

Rôle d'équipe	Contribution transverse
Frontend	Pages Next.js, composants, responsive, thèmes, formulaires, Maquettes et clients API.
Backend	Microservices Go/Gin, routes REST, middlewares, modèles, logique métier et schémas.
Intégration	API Gateway, Docker Compose, variables d'environnement, WebSockets et communication interservices.
Données	Schémas PostgreSQL, collections MongoDB, bucket MinIO, validations et migrations idempotentes.
Qualité	Tests unitaires, Vitest, workflows CI, Swagger, build, lint et validations manuelles.
UI/UX	Navigation inspirée de X/Twitter, mobile-first, thème clair/sombre, accessibilité visuelle et maquettes.

Tableau 12 : Contributions transverses par domaine

Cette répartition a permis à l'équipe de travailler en parallèle tout en gardant un fil conducteur commun : l'utilisateur doit pouvoir passer de l'inscription au fil d'actualité, puis aux interactions sociales et aux paramètres sans rupture fonctionnelle.

6.5 Gestion des risques et des retards

Le suivi des issues a rendu visibles les difficultés et les retards, ce qui a permis de les traiter explicitement plutôt que de les subir. Les principaux retards constatés concernaient des sujets transverses et techniquement sensibles, traités en phase de stabilisation.

Risque / retard	Cause	Mesure prise	Impact résiduel
Liaison des services en production	Différences de configuration réseau entre dev et VPS	Reprise de la configuration gateway et des variables d'environnement	Résolu
Lenteur de la première connexion	Démarrage à froid des conteneurs et seeds	Optimisation du démarrage et seeds de données	Résolu
Stabilisation messagerie E2EE	Combinaison chiffrement client + WebSocket + reçus	Itérations dédiées, refonte UX des groupes	Résolu, audit crypto recommandé
Détection de langue de la traduction	Sur-détection de l'anglais	Correction de la détection + multi-langues	Résolu
Bugs responsive et navigateurs Chromium	Différences d'affichage mobile/desktop	Campagne de corrections UI ciblée	Résolu
Délai d'attribution du rôle administrateur	Propagation des claims dans le JWT	Diagnostic et correction du flux de rôle	Résolu

Tableau 13 : Gestion des risques et des retards

La stratégie de maîtrise des risques reposait sur trois leviers : (1) la conteneurisation pour garantir la reproductibilité entre environnements, (2) la CI pour détecter tôt les régressions et la dérive de documentation, et (3) une phase de stabilisation dédiée en fin de projet pour absorber la dette accumulée pendant les développements rapides. Les rares tâches non terminées (quelques tests d'intégration, durcissement sécurité) ont été consciemment

basculées en backlog et documentées comme axes d'amélioration, plutôt que livrées de manière instable (vocaux et page de chargement par exemple).

6.6 Ressources mobilisées

Les ressources techniques principales sont le dépôt de code mono-repo, les environnements Docker, les fichiers .env.example, les Makefiles, la documentation OpenAPI, les workflows GitHub Actions, les tests automatisés et un VPS d'hébergement pour la mise en production. Les ressources humaines ont été réparties par responsabilités plutôt que par modules strictement figés, afin de favoriser l'entraide et la continuité en cas de blocage. L'effort cumulé représente l'équivalent de quatre personnes mobilisées à temps plein sur la durée du projet, soit la base de la valorisation financière présentée en section 12.2.

7. Description détaillée des fonctionnalités principales

7.1 Authentification, comptes et sécurité

Le module d'authentification est porté par auth-service. L'inscription crée un compte avec e-mail, username et mot de passe haché avec bcrypt. Le login accepte l'e-mail ou le username ; dans le cas du username, l'identifiant est résolu via user-service avant vérification des credentials. Le système utilise un access token JWT de courte durée (5 minutes) et un refresh token opaque (24 heures), stocké côté serveur sous forme hachée et renouvelé lors du rafraîchissement de session.

La figure ci-dessous détaille le parcours complet, de la connexion par username jusqu'au rafraîchissement silencieux : le BFF résout l'identifiant et pose le cookie refresh httpOnly, l'API Gateway valide le jeton « si présent » et propage l'identité aux services.

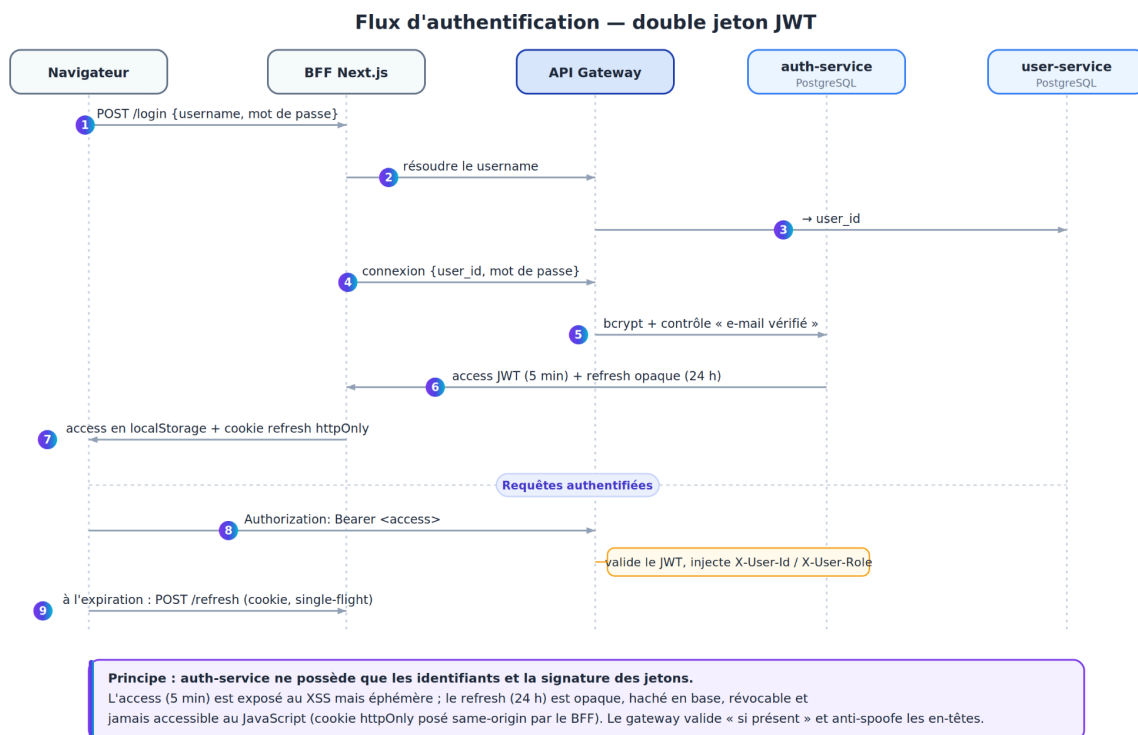


Figure 4 : Flux d'authentification à double jeton JWT

La vérification d'e-mail est obligatoire dans le parcours standard. Lors de l'inscription, auth-service demande à mail-service d'envoyer un lien de confirmation. Les routes de mot de passe oublié et de réinitialisation suivent le même principe avec un token à usage unique. Le projet inclut aussi la connexion sociale OAuth via un registry générique de providers : Google et GitHub, un changement d'e-mail vérifié et une déconnexion qui révoque le refresh token de manière best-effort.

Techniquement, le refresh token est protégé par un cookie httpOnly same-origin posé par le BFF Next.js, tandis que l'access token est utilisé pour les appels Bearer vers la gateway. Cette séparation limite l'exposition du jeton long. Les rôles User, Moderator et Administrator sont portés dans le JWT et utilisés dans l'interface comme dans les services. Les modérateurs accèdent au centre de modération. Les administrateurs disposent d'actions supplémentaires comme la création de compte. Lorsqu'un administrateur crée un compte, l'utilisateur reçoit un mot de passe temporaire par e-mail ; à la première connexion, une modale bloquante impose le changement de mot de passe (drapeau must_change_password), et un username déjà pris est provisoirement suffixé puis

régularisé au premier vrai renommage (username_pending). Ces drapeaux transitent par le JWT, ce qui évite un appel serveur supplémentaire.

Cette [matrice de droits par rôle](#) répond directement à l'exigence de routes protégées et de sessions sécurisées du cahier des charges.

Pour une première connexion sociale, aucun compte n'est créé tant que l'utilisateur n'a pas saisi un username, une date de naissance et accepté les CGU sur une page dédiée : avant cette étape, seule une demande d'inscription temporaire (valable 15 minutes) existe, sans aucun jeton de session. Il est ainsi impossible de créer un compte sans consentement explicite, ce qui répond à l'exigence légale (RGPD).

7.2 Publications et fil d'actualité

Les posts sont gérés par post-service et stockés dans MongoDB. Une publication est limitée à 280 caractères et peut inclure des médias, des emojis, des hashtags, un sondage, une visibilité et une audience de réponse. Les médias sont représentés par des références vers media-service et MinIO, ce qui évite de stocker les fichiers directement dans MongoDB.

La limite pour les médias est de 5 Mo pour les images et 50 Mo pour les vidéos.

Le fil d'actualité propose deux vues principales : Pour toi et Abonnements . Les contenus sont chargés avec pagination infinie, ce qui évite de récupérer tout le fil en une seule requête. Les posts sont filtrés selon la visibilité des profils, les relations de follow et les règles d'audience. Un bandeau temps réel signale les nouveaux posts publics grâce à un WebSocket exposé par post-service et relayé par la gateway.

Les interactions incluent les likes, commentaires, réponses à deux niveaux, reposts, citations, pins et signets. Les commentaires respectent l'audience de réponse configurée sur le post. Les signets sont organisables en collections, avec une collection par défaut qui ne peut pas être supprimée. Les résultats de sondage et certaines règles de visibilité sont appliqués côté serveur, afin que la sécurité ne dépende pas seulement de l'interface.

Le partage de publications et de profils est entièrement géré côté client (aucun service dédié) via un composant ShareDialog commun, qui propose trois voies : l'envoi par message privé, le lien voyage alors dans un message chiffré de bout en bout, le partage natif du

système (navigator.share) et la copie du lien. Dans une conversation, un lien partagé est ré-enrichi en carte cliquable (aperçu du post ou du profil).

Breezy gère le contenu sensible (NSFW) avec une barrière de majorité calculée côté serveur. L'âge est dérivé de la date de naissance (≥ 18 ans) à la lecture, jamais stocké comme un drapeau : un compte se débloquent automatiquement le jour de ses 18 ans. Un post peut être marqué sensible par son auteur ou par un modérateur/administrateur ; le serveur reste l'autorité (nsfw_visible = majeur ET préférence activée), le front se contentant de flouter l'aperçu. Le seuil de majorité (18 ans) est distinct de l'âge minimum d'inscription (13 ans). C'est un nouvel exemple du principe « la sécurité ne dépend jamais du front ».

7.3 Profils et relations sociales

Le profil utilisateur est séparé de l'identité technique. user-service possède le username, le statut actif, le rôle et le graphe social, tandis que profil-service possède les informations décoratives : nom affiché, bio, avatar, bannière, localisation, site web, nationalité, genre, date de naissance et visibilité. Cette séparation limite les responsabilités de chaque service et rend le modèle plus clair.

Les utilisateurs peuvent suivre d'autres comptes. Si un profil est privé, la demande de follow reste en attente jusqu'à acceptation ou refus. Les listes d'abonnés et d'abonnements sont disponibles, ainsi que des suggestions de profils. Le profil affiche les publications et les réponses de l'utilisateur. L'onglet « J'aime » d'un profil est régi par un paramètre de visibilité dédié : l'utilisateur décide si ses likes sont publics ou masqués, indépendamment de la visibilité de son profil.

Breezy implémente également le blocage d'utilisateur. Le blocage est une relation du graphe social, donc porté par user-service (table blocks, à côté des follows et des demandes de suivi). Son effet est appliqué côté serveur dans post-service : la fonction canReadAuthor exclut les contenus du bloqueur comme du bloqué sur toutes les lectures (fil, détail d'un post, statistiques, profil, réponses, likes), si bien qu'aucun client ne peut contourner le filtre. Le déblocage rétablit les règles normales mais ne recrée pas les abonnements supprimés ; un profil redevenu privé reste soumis à l'acceptation de la demande de suivi.

Breezy affiche aussi le statut de présence : un utilisateur peut activer le suivi de son activité, ce qui montre un indicateur « en ligne » ou « vu pour la dernière fois il y a X ». Cette donnée appartient à profil-service, à côté de la visibilité, avec son propre interrupteur de confidentialité : un profil privé ne révèle son activité qu'à son propriétaire et à ses abonnés acceptés, et l'information est omise si l'utilisateur l'a désactivée. Un simple rafraîchissement de session ne compte pas comme une nouvelle connexion.

7.4 Explorer, recherche, hashtags et tendances

La page Explorer regroupe les tendances, les suggestions de comptes et la recherche. L'utilisateur peut rechercher par hashtag, username ou nom affiché. Les filtres permettent de distinguer les publications des utilisateurs. Les hashtags sont extraits côté serveur, dénormalisés sur les posts, affichés comme éléments cliquables et utilisés pour produire des tendances visibles dans le fil et la colonne droite.

Un historique de recherche côté frontend améliore la navigation sans ajouter une complexité excessive au back-end. La recherche full-text avancée reste toutefois une amélioration à prévoir, car la version actuelle est adaptée au périmètre projet mais moins puissante qu'un moteur spécialisé.

7.5 Notifications temps réel

notification-service reçoit des événements internes liés aux likes, commentaires, réponses, mentions, follows, demandes de follow, acceptations de follow, reposts, citations et mentions dans les messages. Les notifications sont agrégées pour éviter de saturer l'utilisateur avec des éléments trop répétitifs. Le frontend maintient un badge de notifications et une page dédiée.

La communication temps réel utilise WebSocket. Lorsqu'un événement concerne un utilisateur connecté, la notification peut être poussée sans attendre un rechargement de page. Les événements internes sont envoyés en best-effort par les services métier vers notification-service à travers une route interne protégée par un secret, ce qui évite d'exposer cette ingestion au client.

Un point d'architecture mérite d'être souligné : Breezy ne généralise pas WebSocket par défaut, mais choisit le mécanisme temps réel selon l'échelle. Les notifications et le bandeau « nouveau post » passent par WebSocket, car ce sont des événements ciblés et peu fréquents. En revanche, les compteurs de likes et de commentaires sont rafraîchis par interrogation périodique groupée (une requête GET /posts/stats?ids=... toutes les sept secondes, portant uniquement sur les posts affichés à l'écran). Pousser chaque like de chaque post public vers toutes les connexions imposerait une couche de coalescence serveur, coûteuse et inutile à notre échelle. Ce choix d'adapter l'architecture au volume réel plutôt que copier X aveuglément illustre une démarche d'ingénierie raisonnée.

7.6 Messagerie privée chiffrée

La messagerie est l'une des parties les plus avancées du projet. message-service gère les conversations directes, les groupes et les communautés. Le contenu des messages est chiffré de bout en bout côté client à l'aide de clés X25519 et d'un chiffrement XChaCha20-Poly1305. Le serveur conserve les métadonnées nécessaires au fonctionnement, mais le contenu utile reste chiffré : le serveur est volontairement aveugle au corps des messages privés.

Le schéma suivant illustre le principe : tout le chiffrement se déroule dans le navigateur, message-service ne reçoit et ne stocke que des enveloppes, et le destinataire déchiffre localement — le serveur est donc aveugle au contenu (admin-proof), à l'exception assumée des communautés.

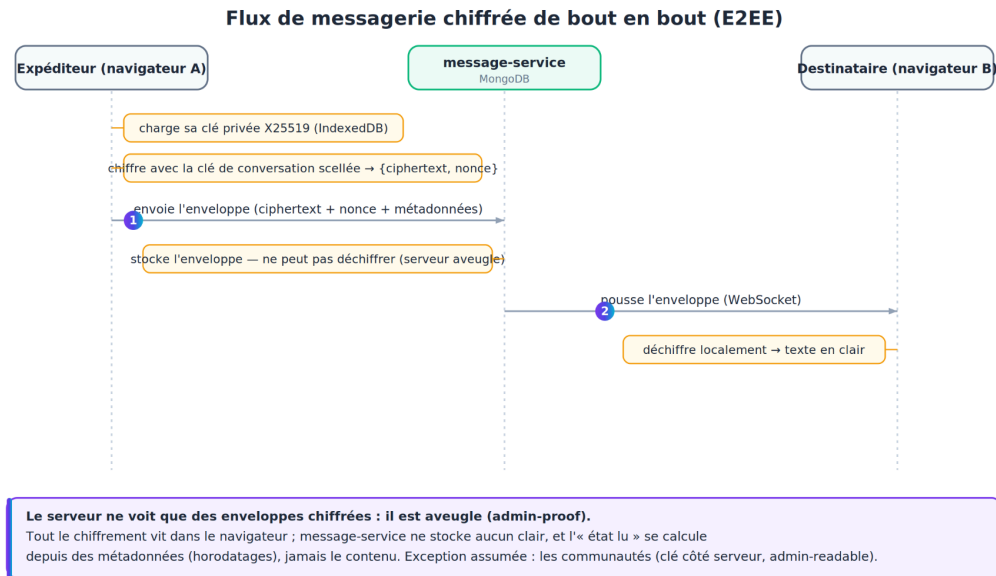


Figure 5 : Flux de messagerie chiffrée de bout en bout (E2EE)

Cette propriété mérite une nuance, qui est aussi un choix d'architecture assumé. Les messages directs et les groupes sont admin-proof : la clé de contenu est scellée pour chaque membre, le serveur ne détient aucune clé et ne peut donc rien déchiffrer. Les communautés, en revanche, sont admin-readable par conception : comme elles admettent un nombre illimité de membres-spectateurs, un chiffrement de bout en bout strict est impossible (il faudrait un détenteur de clé toujours en ligne) ; la clé est conservée côté serveur, sur le modèle d'un canal public type Telegram. Cette distinction est annoncée à l'utilisateur et n'affaiblit pas la confidentialité des conversations privées.

Le module prend en charge les messages temps réel, les pièces jointes chiffrées, les accusés envoyé/ouvert, l'indicateur de saisie, la mise en sourdine, l'édition et la suppression de message. L'identité de chiffrement est associée à l'appareil et peut être sauvegardée avec une clé de secours protégée par passphrase, afin de retrouver l'accès sur un autre appareil sans transmettre la passphrase en clair au serveur.

La messagerie impose donc un compromis : le serveur doit stocker des informations de conversation, d'appartenance et de statut, mais il ne doit pas pouvoir lire le corps des messages. Cette séparation entre contenu chiffré et métadonnées rend le module plus complexe, mais plus proche d'une application réelle.

7.7 Médias et stockage objet

media-service centralise le stockage des avatars, bannières, images, vidéos et pièces jointes chiffrées. Les fichiers sont placés dans MinIO, qui fournit une API compatible stockage objet. Le service vérifie la taille, le type MIME et les droits de suppression. Les médias sont ensuite servis via la gateway, ce qui évite d'exposer directement MinIO aux clients.

Pour les posts, la limite est fixée à quatre médias par publication. Les avatars et bannières sont raccordés au profil, tandis que les pièces jointes de messages utilisent une route dédiée aux blobs chiffrés. Le streaming et les requêtes Range permettent une lecture plus propre des vidéos, avec une limite de taille (5Mo pour les images et 50 Mo pour les vidéos) et une compression côté upload. Les médias sont volontairement traités comme des objets opaques : media-service ne décide pas du sens métier du fichier, il gère son stockage et son accès.

Côté lecture, les médias d'un post s'ouvrent dans une visionneuse plein écran (lightbox). Les vidéos du fil démarrent en lecture automatique muette lorsqu'elles deviennent visibles à l'écran et se mettent en pause hors champ, à la manière de X ; les vidéos situées loin dans le fil ne sont ni téléchargées ni lues tant qu'on ne les approche pas.

L'ajout de GIFs animés (recherche GIPHY) est intégré : media-service expose un proxy vers l'API GIPHY, la clé restant côté serveur et jamais exposée au navigateur. Le GIF choisi n'est pas hotlinké depuis le CDN externe mais capturé dans MinIO et re-servi en same-origin, ce qui le rend compatible avec la politique de sécurité de contenu (CSP self) et insensible aux pannes du CDN. Un filtrage par liste blanche d'hôtes (anti-SSRF) borne les domaines appelables.

7.8 Paramètres, thème et internationalisation

La page Paramètres regroupe la gestion du compte, la confidentialité, la langue, le thème et les mots masqués. L'utilisateur peut changer son mot de passe, demander un changement d'e-mail vérifié, modifier la visibilité de son profil et de ses likes, choisir un thème clair ou sombre et personnaliser les couleurs via une roue chromatique maison (sans dépendance

externe), agissant sur trois cibles indépendantes : l'arrière-plan, le texte et la couleur primaire. Cette dernière avec un contraste recalculé automatiquement pour rester lisible.

L'internationalisation prend en charge douze langues : français, anglais, chinois, espagnol, portugais, russe, japonais, coréen, arabe, hindi, allemand et italien. La préférence est persistée par compte, les dates sont localisées et l'arabe active le support RTL. Les posts et commentaires peuvent être traduits prudemment via une route BFF, avec détection de langue et cache côté application.

7.9 Modération et administration

Le signalement de contenu est porté par un service dédié, report-service. Depuis n'importe quelle publication, message ou profil, un utilisateur peut signaler un contenu en choisissant un motif (inapproprié, offensant, spam, bug ou autre). Chaque signalement crée ou enrichit un ticket de modération : les modérateurs disposent d'une file de tickets avec réponses, changement de statut, transfert, validation et retrait du contenu visé. Un mécanisme d'auto-masquage retire automatiquement un post lorsqu'il dépasse un seuil de signalements configurable, et un système d'avertissements notifie l'utilisateur fautif, qui doit en accuser réception. Le dispositif reste compatible avec la messagerie chiffrée : l'utilisateur divulgue volontairement le message ciblé au moment du signalement, sans casser le chiffrement de bout en bout des autres messages.

Le centre de modération est accessible aux modérateurs et administrateurs. Il comprend une corbeille de posts supprimés, avec possibilité de restaurer un contenu ou de le purger définitivement. Les comptes peuvent être bannis, réactivés ou encore averti (message affiché sur l'écran). Un modérateur agit sur la sécurité de la communauté, tandis qu'un administrateur possède des droits supplémentaires de gouvernance.

La figure ci-dessous synthétise le cycle : un signalement est agrégé dans un ticket par report-service, l'auto-masquage est déclenché par un appel serveur-à-serveur vers post-service (hors gateway, authentifié par secret partagé), et les trois rôles se répartissent les responsabilités — l'utilisateur signale, le modérateur traite, l'administrateur gouverne.

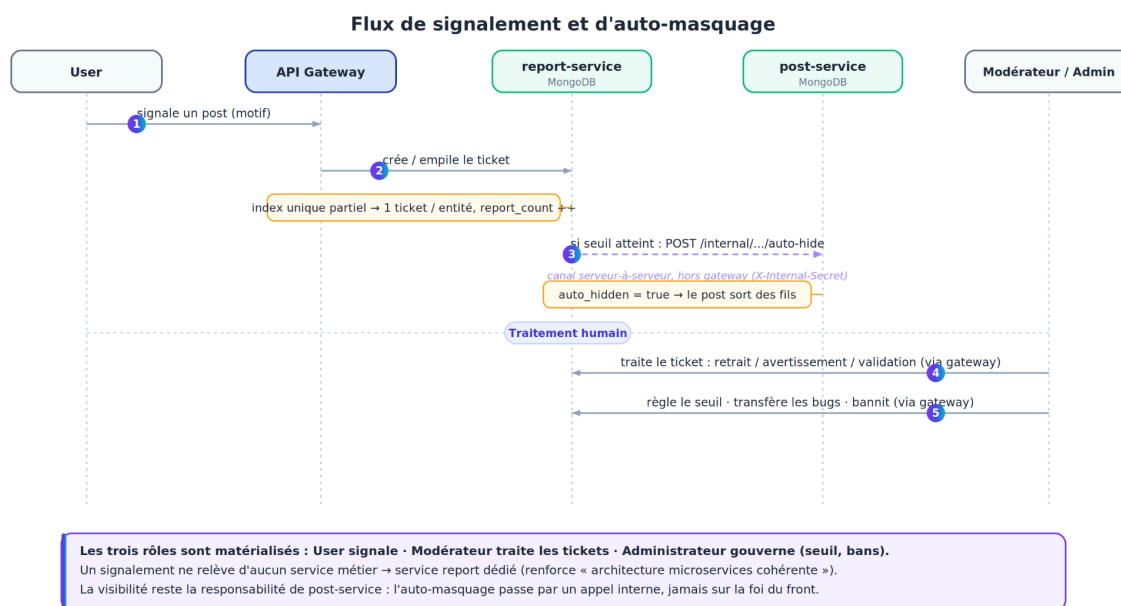


Figure 6 : Flux de signalement et d'auto-masquage

La page d'administration est réservée aux administrateurs. Elle affiche un état des services avec statut, latence et uptime via une route de monitoring de l'API Gateway. Elle permet également la création d'un compte par administrateur avec mot de passe temporaire. Cette séparation entre modération et administration clarifie les rôles : la modération traite les contenus et comptes, l'administration surveille l'infrastructure et les actions de gouvernance.

7.10 Mode visiteur

Un utilisateur non connecté accède au fil public sans inscription. La lecture publique est servie par post-service en authentification optionnelle : sans jeton, la barrière de visibilité serveur ne laisse sortir que les posts publics. L'interface réutilise le même fil que l'utilisateur connecté, avec une barre latérale réduite (boutons Se connecter / S'inscrire). Les actions réservées (aimer, republier, commenter, signaler) n'entraînent pas une redirection brutale vers la connexion, mais ouvrent une invitation « Connecte-toi pour... », ce qui laisse le visiteur explorer sans friction. Ce mode concrétise le rôle Visiteur du cahier des charges tout en restant une lecture seule strictement contrôlée côté serveur.

7.11 Matrice des rôles

La matrice ci-dessous récapitule les droits par rôle. Breezy implémente trois rôles authentifiés (User, Modérateur, Administrateur) et un mode visiteur en lecture seule sur le contenu public.

Légende :  action autorisée ·  interdit.

Fx	Fonctionnalité	Visiteur	User	Modérateur	Admin
Fx1	Création de compte				
Fx2	Authentification				
Fx3	Publication de posts				
Fx4	Affichage des posts sur profil				
Fx5	Fil d'actualité (public)				
Fx6	Liker				
Fx7	Commenter un post				
Fx8	Répondre à un commentaire				
Fx9	Suivre / être suivi				
Fx10	Profil utilisateur				
Fx11	Liste des posts sur profil				
Fx12	Hashtags				
Fx13	Recherche par tags				
Fx14	Notifications de mention				

Fx15	Notifications de likes	✗	✓	✓	✓
Fx16	Notifications de followers	✗	✓	✓	✓
Fx17	Messages privés (E2EE)	✗	✓	✓	✓
Fx18	Images	✗	✓	✓	✓
Fx19	Vidéos	✗	✓	✓	✓
Fx20	Signalement de contenu	✗	✓	✓	✓
Fx21	Suspension / bannissement	✗	✗	✓	✓
Fx22	Interface multilingue	✓	✓	✓	✓
Fx23	Thème personnalisé	✓	✓	✓	✓
	Modération (corbeille, retrait, avertissement)	✗	✗	✓	✓
	Administration (rôles, monitoring, compte admin)	✗	✗	✗	✓

Tableau 14 : Matrice des permissions par rôle

8. Interfaces utilisateur, UX et Maquettes

L'interface de Breezy s'inspire de X/Twitter sans chercher à la copier exactement. Sur desktop, elle utilise une barre latérale gauche pour la navigation, une colonne centrale pour le contenu et une colonne droite pour la recherche, les tendances et les suggestions. Sur mobile, la navigation est simplifiée avec un header compact, une barre d'onglets en bas et un bouton flottant de composition.

Le design est mobile-first. Les composants privilégient la lisibilité, les contrastes, les espacements réguliers et les actions rapidement accessibles. Les effets de glassmorphism et les dégradés Breezy donnent une identité visuelle au projet, mais les écrans restent organisés autour de cartes, d'onglets et de formulaires simples. Les maquettes ci-dessous présentent les écrans principaux ; elles ont guidé la réalisation.

Maquette 1 - Écran d'authentification

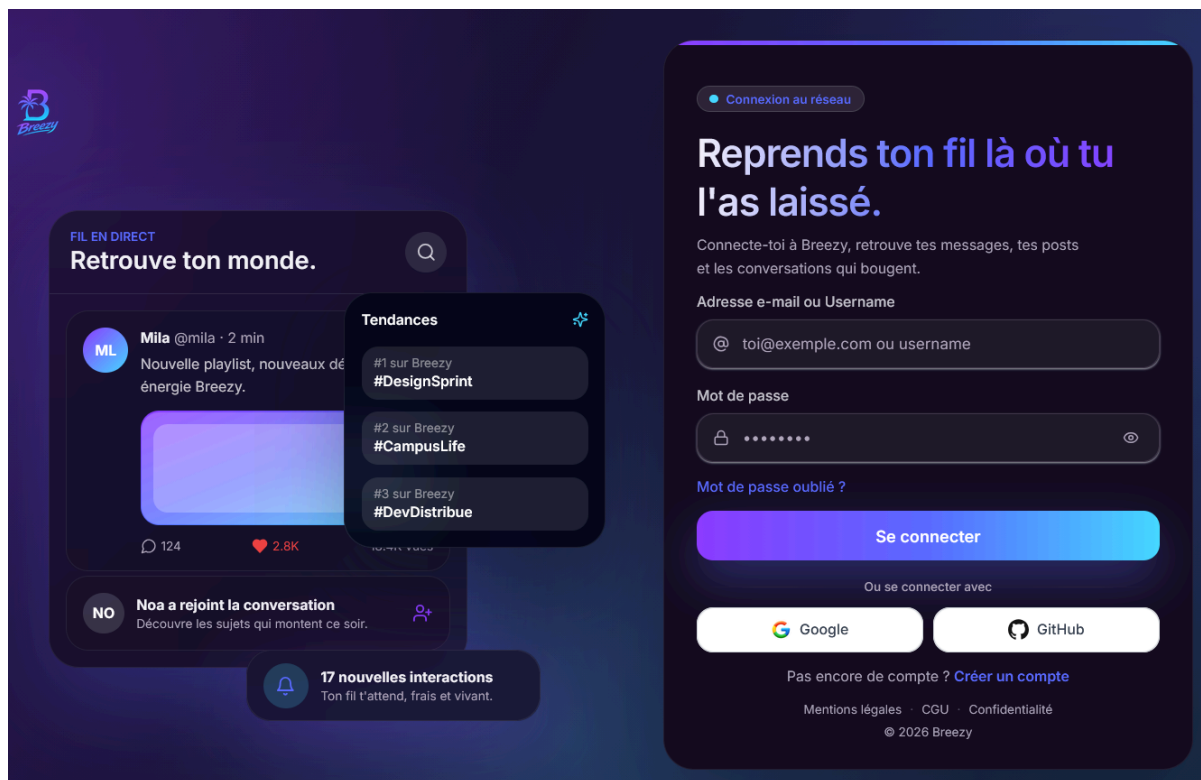


The image shows a wireframe for the authentication screen of the Breezy application. It is divided into two main sections. The left section features the Breezy logo, which consists of a stylized 'B' with a palm tree inside, and the word 'Breezy' in a blue, italicized font. Below the logo, the text 'Réseau social léger mobile-first' is displayed. The right section is titled 'Connexion / Inscription' and contains a form with two input fields: 'Email ou username' and 'Mot de passe'. Below these fields is a blue button labeled 'Se connecter / S'inscrire'. Underneath the button is a white button labeled 'Continuer avec Google'. At the bottom of the right section, there are three links: 'Mot de passe oublié', 'CGU', and 'Confidentialité'.

Maquette 1 - Écran d'authentification

L'écran d'authentification regroupe le logo Breezy, le formulaire de connexion ou d'inscription, les champs e-mail/username et mot de passe, le bouton Google, ainsi que les

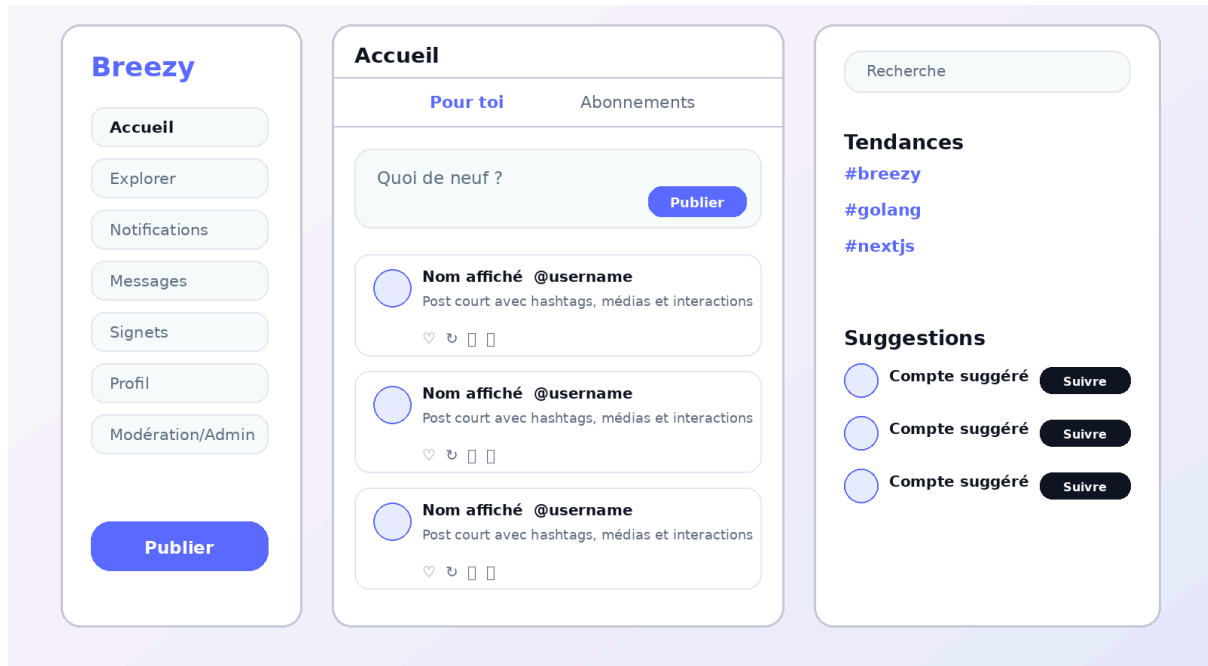
liens mot de passe oublié, CGU et confidentialité. Le choix UX est de rassurer l'utilisateur dès l'entrée dans l'application : le parcours est court, les actions principales sont visibles et les liens légaux restent accessibles sans alourdir le formulaire.



Interface 1 - Écran d'authentification

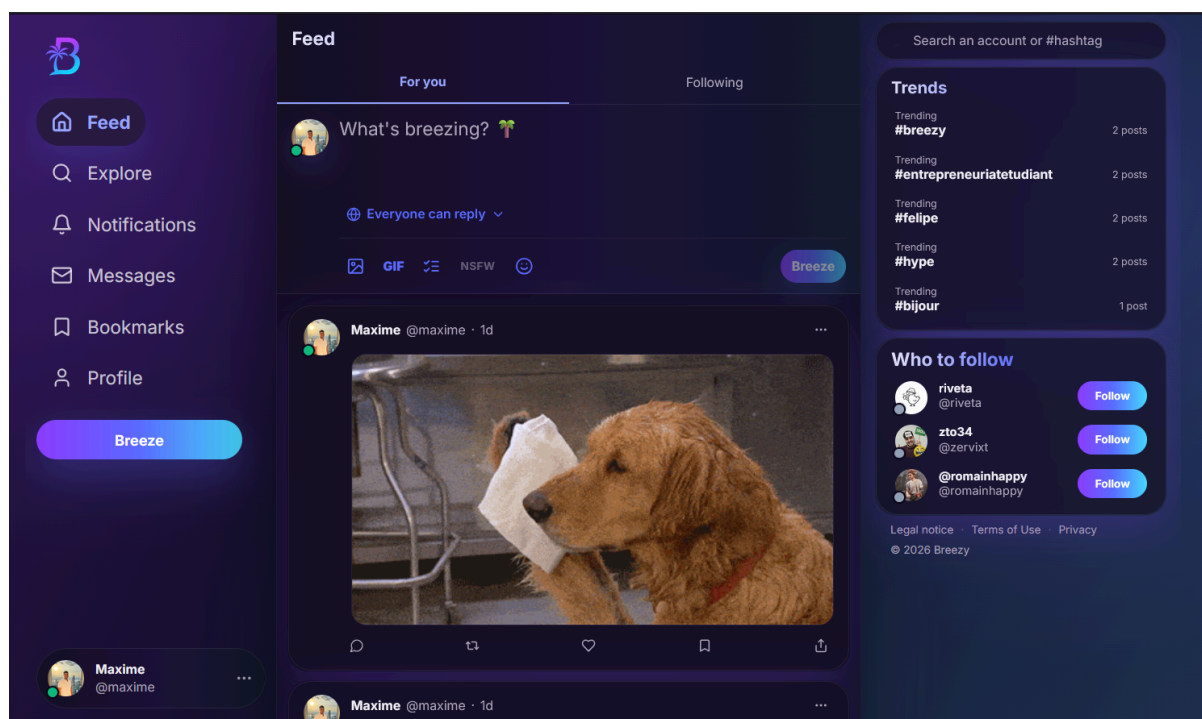
Par rapport à la maquette initiale, l'interface finale est beaucoup plus aboutie visuellement et plus proche d'une vraie application moderne. La maquette servait surtout à poser les éléments principaux, comme le logo, les champs de connexion, le bouton Google et les liens utiles. On a aussi séparé la partie authentification de la partie présentation de Breezy : à gauche, l'utilisateur comprend directement l'ambiance du réseau social grâce au fil en direct, aux tendances et aux notifications, tandis qu'à droite il peut se connecter clairement. Cela rend l'écran plus vivant et plus rassurant, car on ne voit pas seulement un formulaire, mais aussi un aperçu concret de ce que l'application propose. L'interface finale est donc plus professionnelle, plus attractive et elle donne davantage envie d'utiliser Breezy dès la première page.

Maquette 2 - Fil d'actualité desktop



Maquette 2 - Fil d'actualité desktop

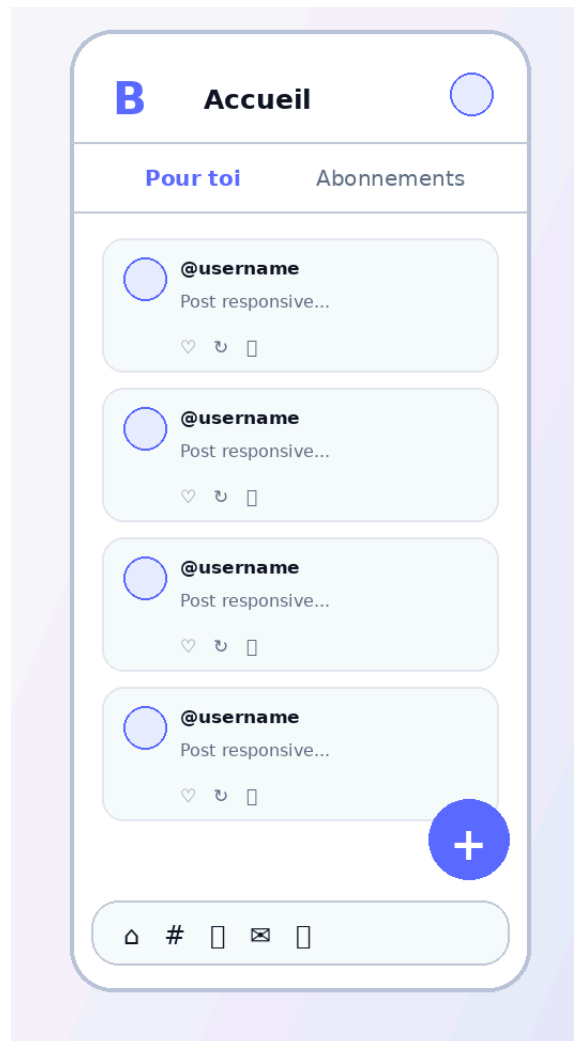
Le fil desktop suit une structure en trois colonnes. La colonne gauche sert de navigation permanente, la colonne centrale contient les onglets Pour toi et Abonnements , la zone de composition et les cartes de posts, tandis que la colonne droite regroupe recherche, tendances et suggestions. Cette organisation reprend un modèle familier pour les réseaux sociaux et limite les changements de contexte.



Interface 2 - Fil d'actualité desktop

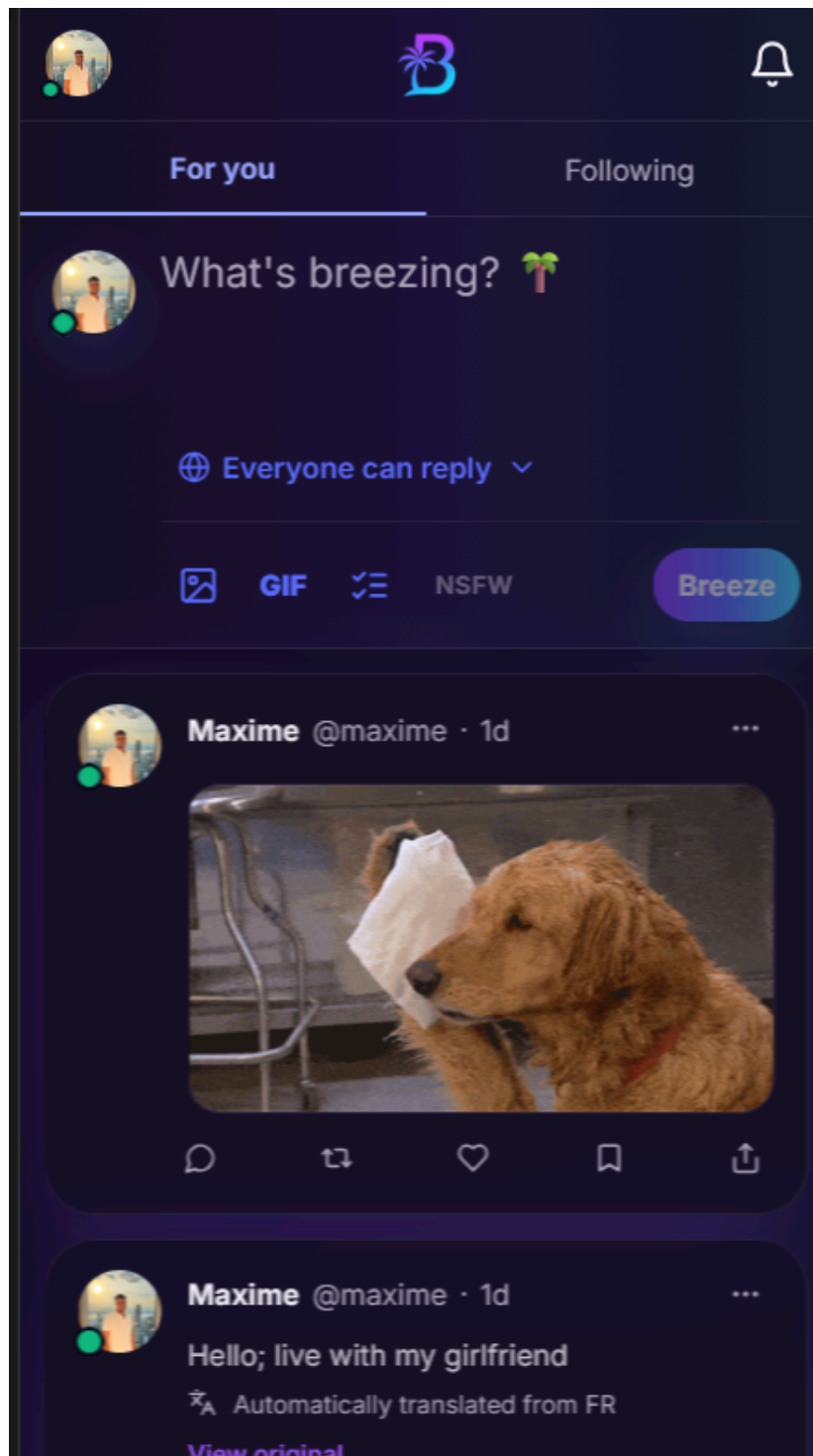
Par rapport à la maquette du feed, l'interface finale est beaucoup plus complète et plus réaliste. La maquette présentait déjà la structure principale avec la barre de navigation à gauche, le fil au centre et les tendances à droite, mais la version développée va plus loin en donnant une vraie ambiance de réseau social. On retrouve un design plus moderne, des icônes plus travaillées et une meilleure hiérarchie visuelle. Le feed est aussi plus vivant : les posts peuvent contenir des médias, on voit les profils, les boutons d'interaction, les tendances avec le nombre de posts et les suggestions de comptes à suivre. L'espace de publication est également plus proche d'une vraie application, avec des options comme les GIF, les images ou le choix de réponse. Au final, l'interface développée garde l'organisation prévue dans la maquette, mais elle l'améliore en rendant l'expérience plus immersive, plus professionnelle et plus agréable à utiliser.

Maquette 3 - Fil d'actualité mobile



Maquette 3 - Fil d'actualité mobile

Sur mobile, la priorité est donnée au contenu. Le header affiche les éléments essentiels, le bouton flottant + permet de publier rapidement et la barre de navigation en bas reste atteignable au pouce. Cette approche mobile-first évite de reproduire une interface desktop trop dense sur petit écran.



Interface 3 - Fil d'actualité mobile

Livable - 25/06/2026

Par rapport à la maquette mobile du feed, l'interface finale est beaucoup plus travaillée et adaptée à une vraie utilisation sur téléphone. La maquette montrait surtout l'organisation générale avec les onglets "Pour toi" et "Abonnements", les posts et la barre de navigation en bas, alors que la version développée propose une expérience plus immersive avec le thème sombre, le logo Breezy centré en haut et des éléments mieux intégrés à l'écran mobile. Le fil est aussi plus réaliste, car on peut voir les profils, les images dans les posts, les boutons d'interaction et même la zone de publication avec les options comme les GIF ou les médias. On a aussi amélioré l'aspect responsive : les éléments prennent bien toute la largeur disponible, les espacements sont plus naturels et l'utilisateur peut comprendre rapidement où publier, consulter son feed ou interagir avec les posts. L'interface finale garde donc la structure pensée dans la maquette, mais elle la rend plus moderne, plus fluide et beaucoup plus proche d'une vraie application sociale mobile.

Maquette 4 - Page profil



Maquette 4 - Page profil

La page profil met d'abord en avant la bannière, l'avatar et les informations personnelles visibles. Les compteurs d'abonnés et d'abonnements sont proches du bouton Suivre ou Modifier le profil. Les onglets Publications, Réponses et Likes permettent de séparer les contenus sans multiplier les pages.



Interface 4 - Page profil

Par rapport à la maquette de la page profil, l'interface finale est beaucoup plus personnalisée et plus proche d'un vrai profil utilisateur. La maquette montrait surtout la

Livrable - 25/06/2026

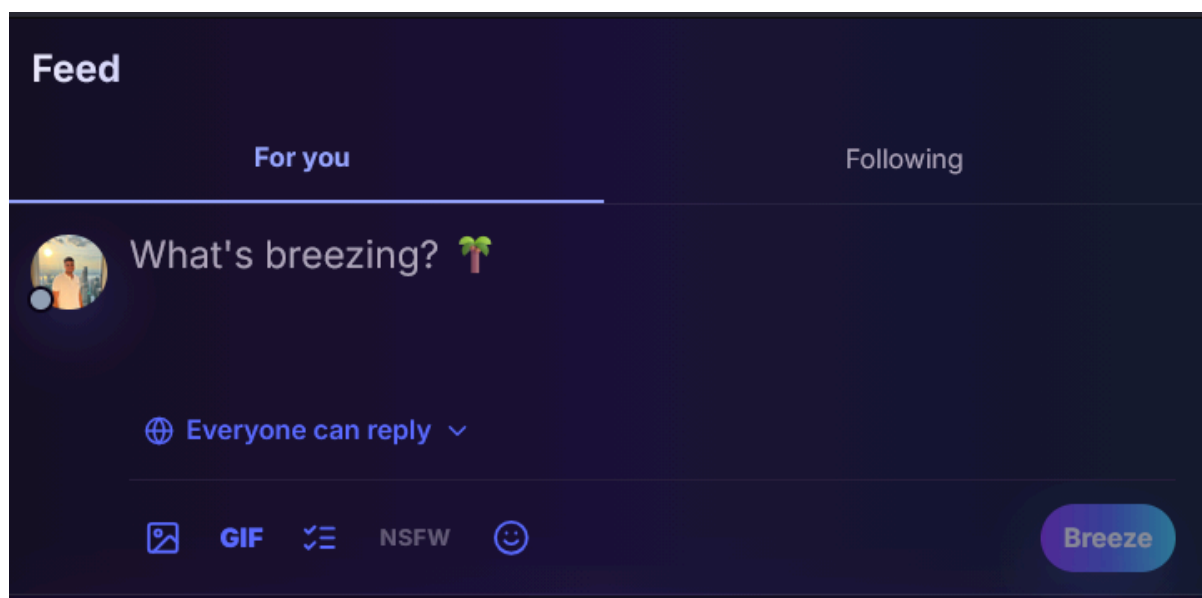
structure générale avec la bannière, la photo de profil, les informations principales, les statistiques et les onglets publications, réponses et likes. Dans la version développée, on a gardé cette base, mais on l'a rendue plus complète avec une vraie photo de couverture, un avatar utilisateur, un statut en ligne, des badges, des informations personnelles comme la localisation, la date de naissance et la date d'inscription. Le design sombre et les couleurs Breezy donnent aussi une identité plus forte à la page. Les publications sont mieux intégrées, avec un affichage plus réaliste des posts, des hashtags, des mentions et des interactions. L'interface finale est donc plus vivante et plus crédible, car elle ne montre pas seulement un profil vide, mais un vrai espace personnel qui donne envie de consulter l'activité de l'utilisateur.

Maquette 5 - Modale de publication

A wireframe of a mobile app's 'Create a post' modal. The modal is a white rounded rectangle on a light purple background. At the top, it says 'Créer une publication'. Below this is a circular profile picture placeholder and a large text input area with the placeholder text 'Quoi de neuf ?'. Under the input area are three blue buttons labeled 'Média', 'Emoji', and 'Sondage'. At the bottom, there is a button labeled 'Qui peut répondre ?', a character count '0 / 280', and a blue 'Publier' button.

Maquette 5 - Modale de publication

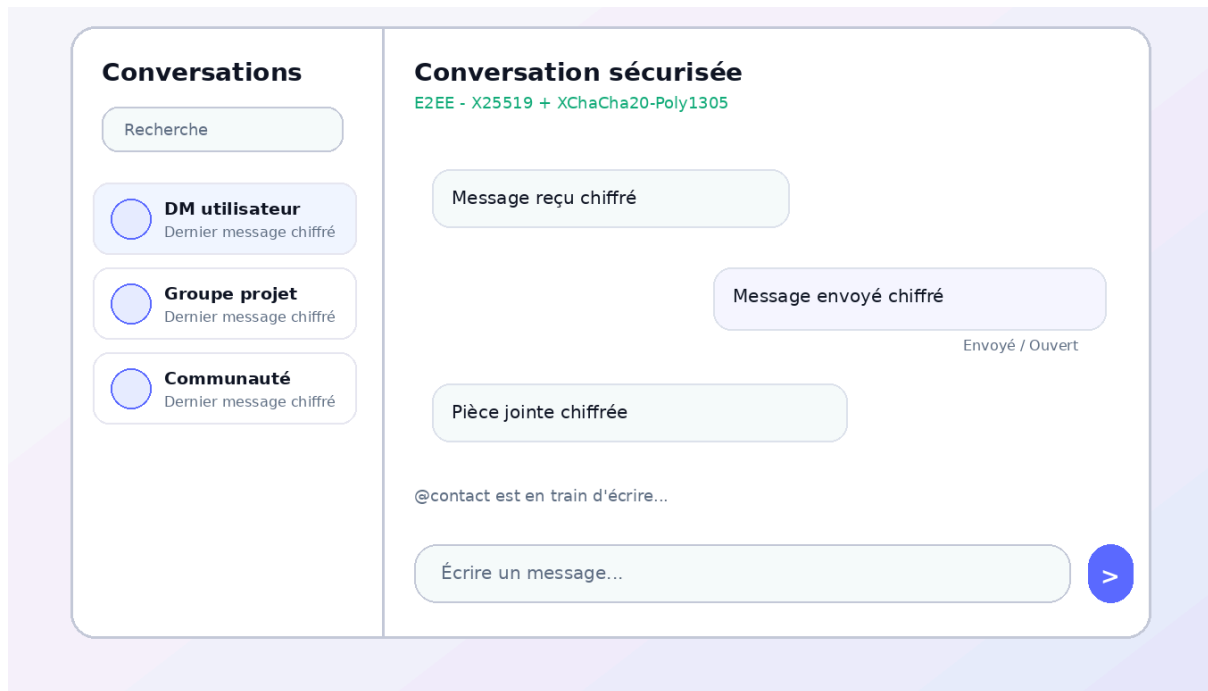
La modale de publication se concentre sur la zone de texte limitée à 280 caractères. Les boutons média, emoji et sondage sont placés près de l'éditeur. Le choix d'audience de réponse et le compteur de caractères rendent les contraintes visibles avant l'envoi, ce qui limite les erreurs utilisateur.



Interface 5 - Modale de publication

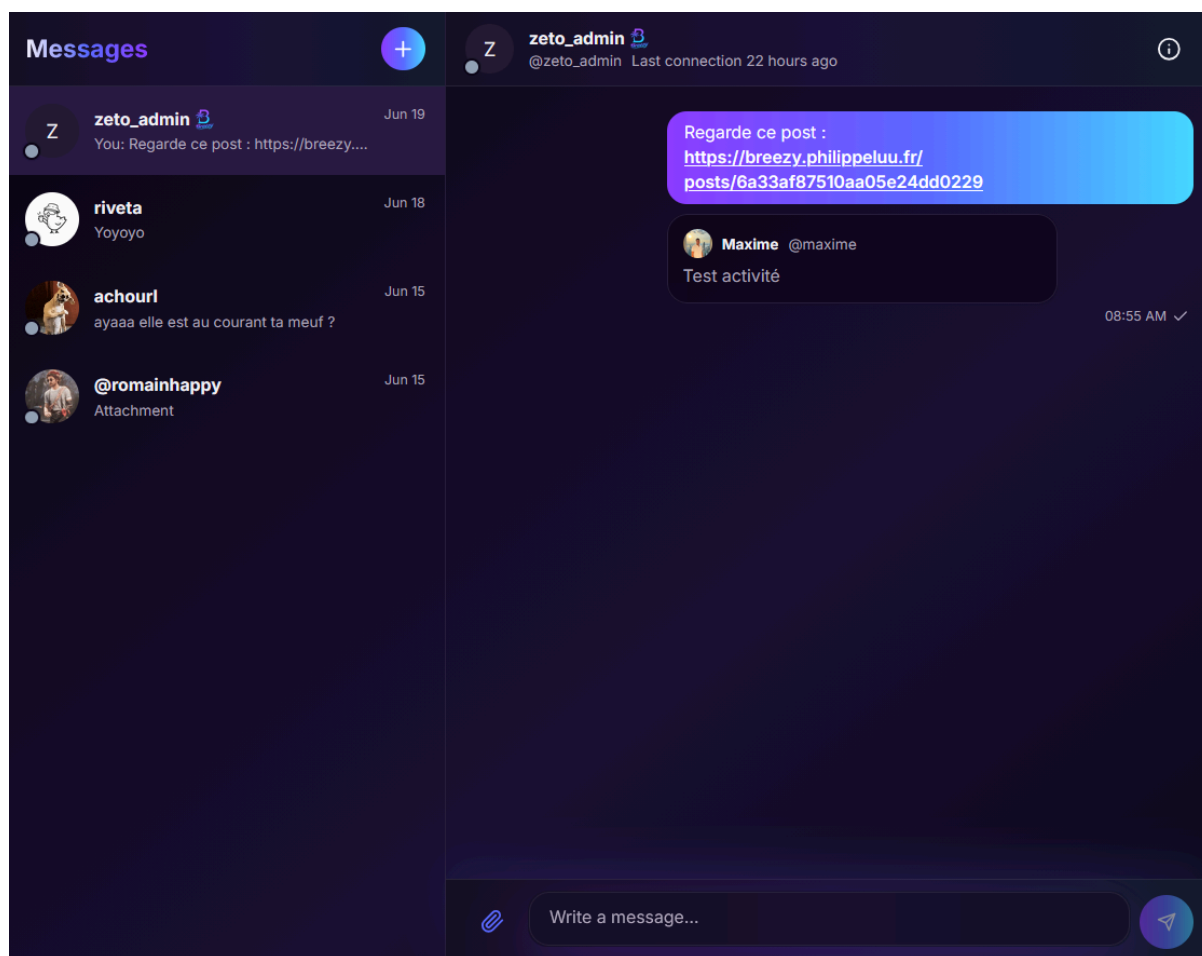
Par rapport à la maquette de création de publication, l'interface finale est plus moderne et mieux intégrée dans le feed. La maquette présentait les éléments essentiels, comme la zone de texte, les options média, emoji, sondage, le choix des réponses et le bouton publier. Dans la version développée, on a gardé cette logique, mais en l'adaptant directement à l'expérience utilisateur principale : l'utilisateur peut écrire son post sans quitter le fil d'actualité. Le design est aussi plus cohérent avec le reste de Breezy grâce au thème sombre, aux couleurs en dégradé et aux icônes plus visuelles. Les options comme l'ajout d'image, de GIF, le sondage ou le choix "Everyone can reply" rendent la publication plus complète et plus proche d'un vrai réseau social. Au final, la version finale est plus fluide, car elle permet de publier rapidement tout en restant dans le contexte du feed.

Maquette 6 - Messagerie



Maquette 6 - Messagerie

La messagerie conserve une séparation classique entre la liste des conversations à gauche et le fil de messages à droite. Les indications de chiffrement, d'envoi, d'ouverture et de saisie rendent le comportement de la conversation compréhensible. Ce choix facilite aussi la transition entre desktop et mobile, où ces zones peuvent devenir deux vues successives.



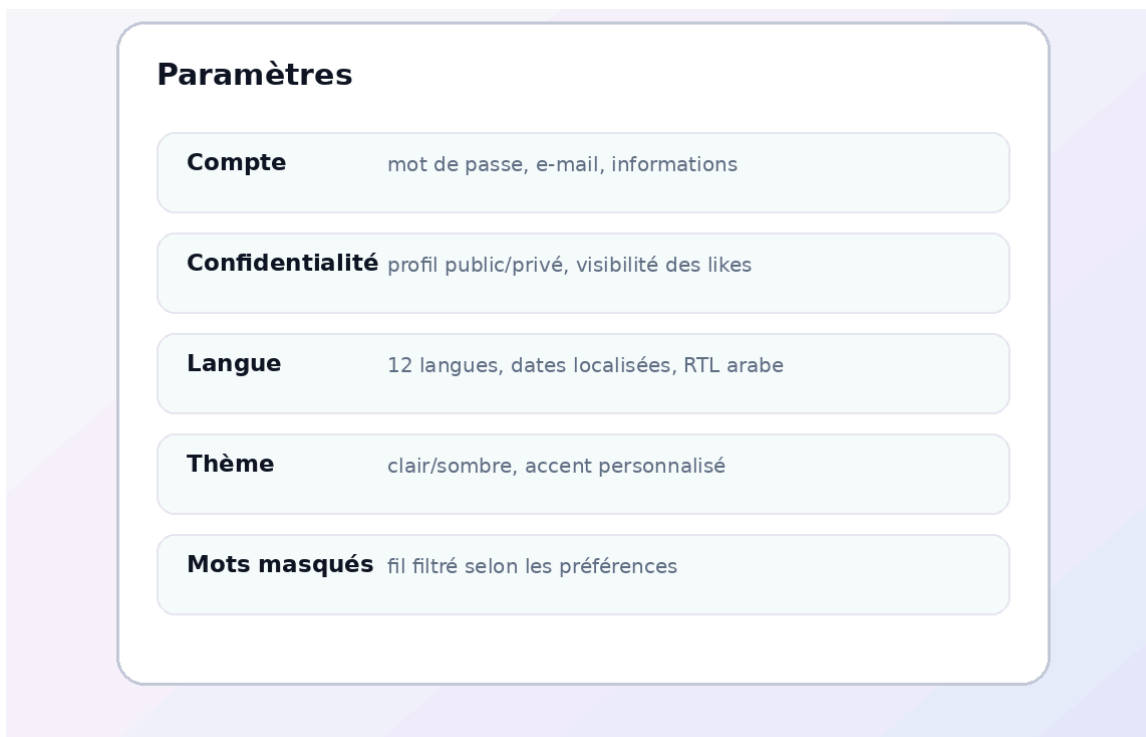
Interface 6 - Messagerie

Par rapport à la maquette de la page messages, l'interface finale est beaucoup plus proche d'une vraie messagerie utilisée dans un réseau social. La maquette montrait surtout le fonctionnement général avec la liste des conversations à gauche, une conversation ouverte à droite, les bulles de messages, les pièces jointes et le champ pour écrire. Dans la version développée, on a gardé cette organisation, mais on l'a rendue plus complète avec un design sombre cohérent avec Breezy, des avatars, les dates des conversations, le statut de connexion et un affichage plus clair du contact sélectionné. Les messages sont aussi plus réalistes, par exemple avec le partage d'un post directement dans la conversation et un aperçu intégré, ce qui rend l'échange plus vivant. Le champ d'écriture en bas, le bouton d'envoi et l'icône de pièce jointe sont mieux placés pour une utilisation rapide. Même si la maquette mettait surtout en avant l'idée de conversation sécurisée, la version finale améliore

Livrable - 25/06/2026

surtout l'expérience utilisateur en donnant une messagerie plus moderne, plus lisible et plus agréable à utiliser.

Maquette 7 - Paramètres



The image shows a wireframe for a 'Paramètres' (Settings) page. It features a central white rounded rectangle with a purple border, set against a light purple background. The title 'Paramètres' is at the top left. Below it are five settings items, each in a light green rounded rectangle: 'Compte' (mot de passe, e-mail, informations), 'Confidentialité' (profil public/privé, visibilité des likes), 'Langue' (12 langues, dates localisées, RTL arabe), 'Thème' (clair/sombre, accent personnalisé), and 'Mots masqués' (fil filtré selon les préférences).

Paramètres	
Compte	mot de passe, e-mail, informations
Confidentialité	profil public/privé, visibilité des likes
Langue	12 langues, dates localisées, RTL arabe
Thème	clair/sombre, accent personnalisé
Mots masqués	fil filtré selon les préférences

Maquette 7 - Paramètres

La page Paramètres est divisée en sections : compte, confidentialité, langue, thème et mots masqués. L'objectif est de regrouper les préférences sensibles sans les cacher dans plusieurs sous-pages. Les actions critiques comme le changement d'e-mail ou de mot de passe restent séparées des réglages visuels.

Settings

**General settings**
Customize your Breezy experience.

**Language**
Choose language

Language

 English 

**Account visibility**
Choose who can see your posts, replies, and likes.

**Private account**
Turn on to limit your content to accepted followers.

**Private likes**
Turn on to hide your likes from other users.

**Hidden activity**
Turn on to hide your online status and last connection.

**Sensitive content**
Manage how content flagged as sensitive (NSFW) is shown.

**Show NSFW content**
Posts marked NSFW are shown without blurring.

**Muted words**
Hide feed posts containing a word or phrase you don't want to see.



No muted words yet.

**Blocked users**
Manage accounts whose posts and activity you no longer see.

 No blocked users.

Interface 7 - Paramètres

 **User settings**
Manage your account credentials and security.

 **Two-factor authentication (2FA)**
Add a one-time code from an authenticator app (Microsoft/Google Authenticator) when you sign in.

Two-factor authentication
Two-factor authentication is not available on this server right now. 

 **Change password**
Changing it will sign out your other sessions.

Current password

New password

Confirm new password

Change password

 **Change email address**
Your current address remains active until the new one is verified.
**maxime.stoffel@viac
esi.fr**

New email address

Send verification link

 **Legal**
[Legal notice](#) · [Terms of Use](#) · [Privacy](#)
© 2026 Breezy

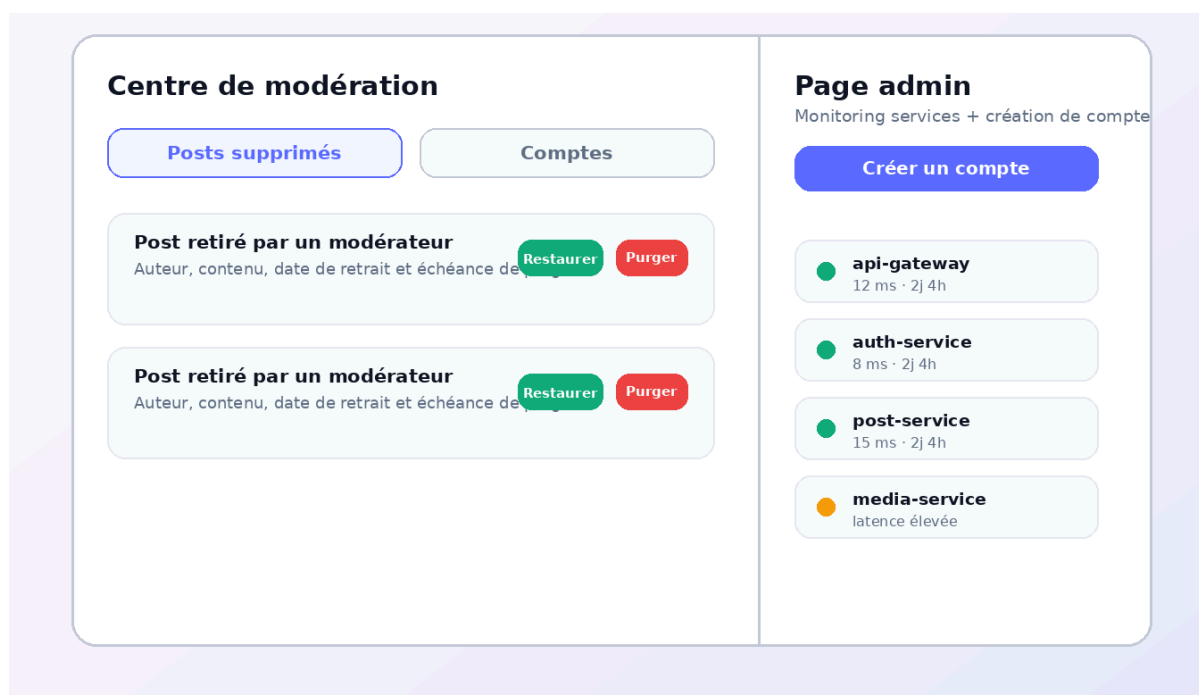
Interface 7 bis - Paramètres

Par rapport à la maquette de la page paramètres, l'interface finale est beaucoup plus détaillée et réellement utilisable. La maquette présentait les grandes catégories importantes, comme le compte, la confidentialité, la langue, le thème et les mots masqués, mais elle

Livrable - 25/06/2026

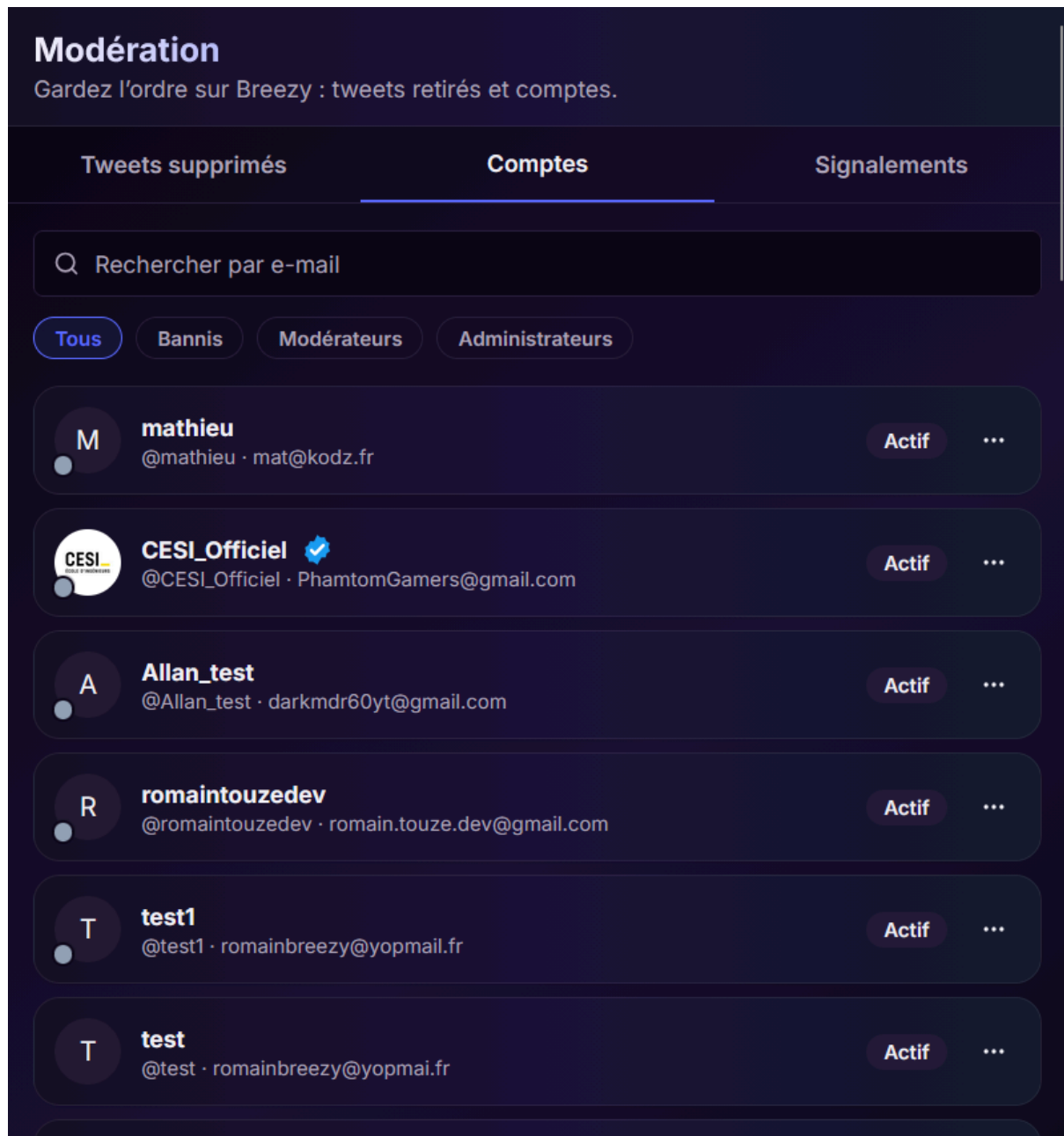
restait assez simple et générale. Dans la version développée, on a gardé cette organisation tout en ajoutant de vraies options configurables, comme le choix de la langue, la visibilité du compte, la possibilité de masquer le contenu sensible, la gestion des mots filtrés et des utilisateurs bloqués. On a aussi ajouté une partie plus complète pour la sécurité du compte, avec l'authentification à deux facteurs, le changement de mot de passe et le changement d'adresse e-mail. Le design sombre est plus cohérent avec le reste de Breezy et les sections sont mieux séparées, ce qui rend la page plus claire malgré le nombre d'options. Au final, la version finale transforme une simple page de réglages en un vrai espace de configuration, plus précis, plus professionnel et plus adapté aux besoins d'un utilisateur.

Maquette 8 - Modération et administration



Maquette 8 - Modération et administration

L'interface de modération distingue les posts supprimés et les comptes. Les actions Restaurer, Purger, Bannir et Réactiver sont visibles mais réservées aux rôles autorisés. La page admin affiche l'état des services séparément, afin de ne pas mélanger les actions de modération de contenu avec le monitoring de l'infrastructure.



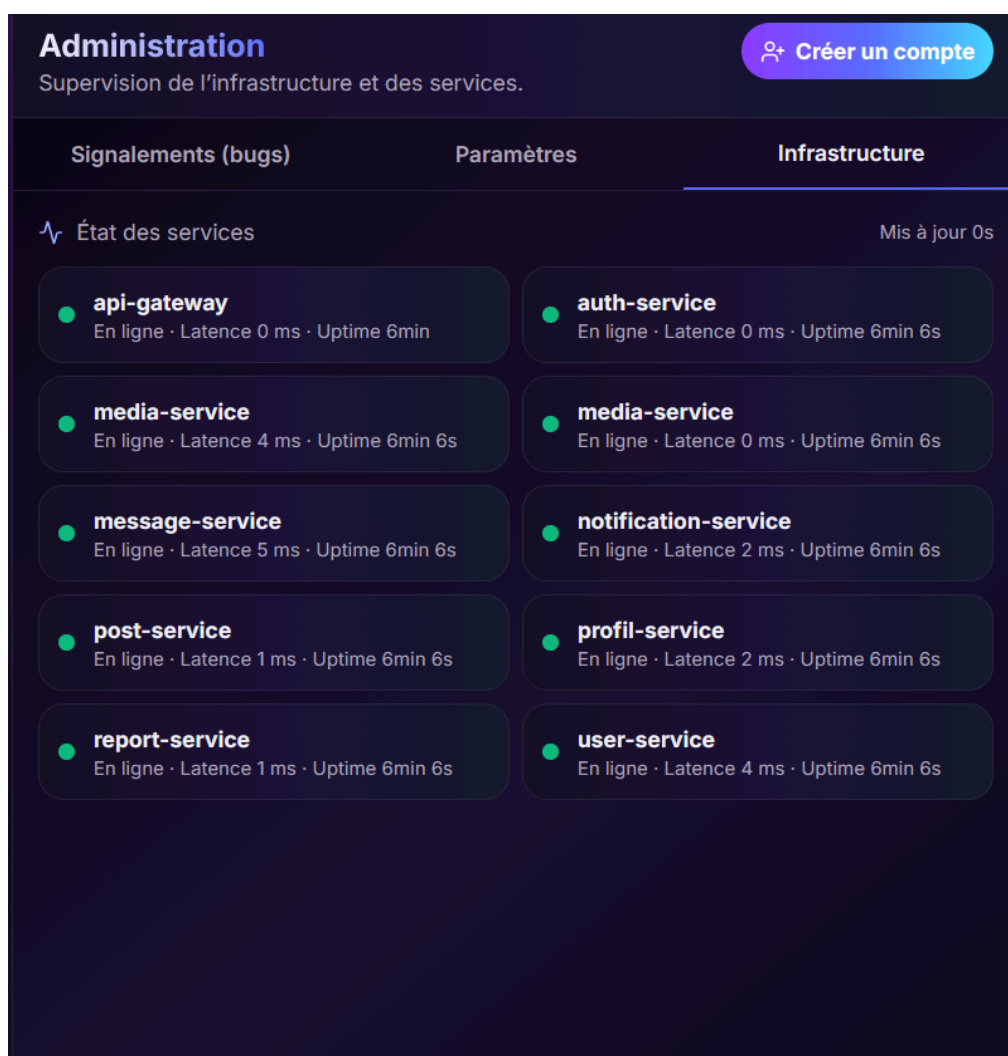
Interface 8 - Modération et administration

Modération :

Par rapport à la maquette, la page de modération finale est beaucoup plus complète et plus pratique pour gérer les utilisateurs. La maquette prévoyait déjà les éléments importants, comme les posts supprimés, les comptes et les actions de restauration ou de suppression. Dans la version développée, on a mieux séparé les différentes parties avec des onglets pour

Livrable - 25/06/2026

les tweets supprimés, les comptes et les signalements. On a aussi ajouté une barre de recherche, des filtres comme “Tous”, “Bannis”, “Modérateurs” et “Administrateurs”, ainsi qu’une liste plus détaillée des comptes avec leur avatar, leur pseudo, leur e-mail et leur statut. Cela rend la modération plus claire et plus efficace, car un modérateur peut trouver rapidement un compte ou une situation à traiter. L’interface finale garde donc l’idée de base de la maquette, mais elle la rend plus professionnelle et plus adaptée à une vraie gestion d’une plateforme sociale.



Interface 8 bis - Modération et administration

Administration :

Pour la partie administration, la version finale est aussi plus avancée que la maquette. Dans la maquette, l'administration était affichée sur le côté avec quelques services à surveiller et un bouton pour créer un compte. Dans l'interface développée, on a créé une vraie page dédiée à la supervision, avec des onglets comme signalements, paramètres et infrastructure. La partie infrastructure est beaucoup plus détaillée, car elle affiche plusieurs services comme l'api-gateway, l'auth-service, le post-service ou encore le message-service, avec leur état, leur latence et leur temps d'activité. Cela donne une vision plus précise de la santé technique de l'application. Le bouton "Créer un compte" est aussi mieux intégré dans l'en-tête, ce qui rend l'action plus visible. Au final, la page administration ne sert plus seulement à afficher quelques informations, elle devient un vrai tableau de bord pour suivre le fonctionnement de Breezy.

9. Tests, qualité, déploiement et documentation

La qualité du projet repose sur plusieurs niveaux de validation. Les services Go contiennent des tests unitaires portant sur des règles métier, des middlewares et certaines fonctions critiques ; la couverture a été poussée au-delà de 95 % sur plusieurs services. Le frontend utilise Vitest pour tester des fonctions utilitaires (gestion des couleurs, des pays, des mentions, des hashtags, de la cryptographie ou des traductions), la couverture de la couche lib ayant également dépassé 95 %. Ce coverage est accessible via [Codecov](#).

api-gateway/internal	250	244	1	5		97.60%
auth-service/internal	2128	2001	50	77		94.03%
frontend/src/lib	20799	19779	46	974		95.10%
mail-service/internal	186	186	0	0		100.00%
media-service/internal	786	748	16	22		95.17%
message-service/internal	1761	1689	34	38		95.91%
notification-service/internal	737	720	5	12		97.69%
post-service/internal	3132	2963	78	91		94.60%
profil-service/internal	830	769	25	36		92.65%
report-service/internal	790	771	9	10		97.59%
user-service/internal	1169	1089	35	45		93.16%

Figure 7 : Coverage des services (Codecov)

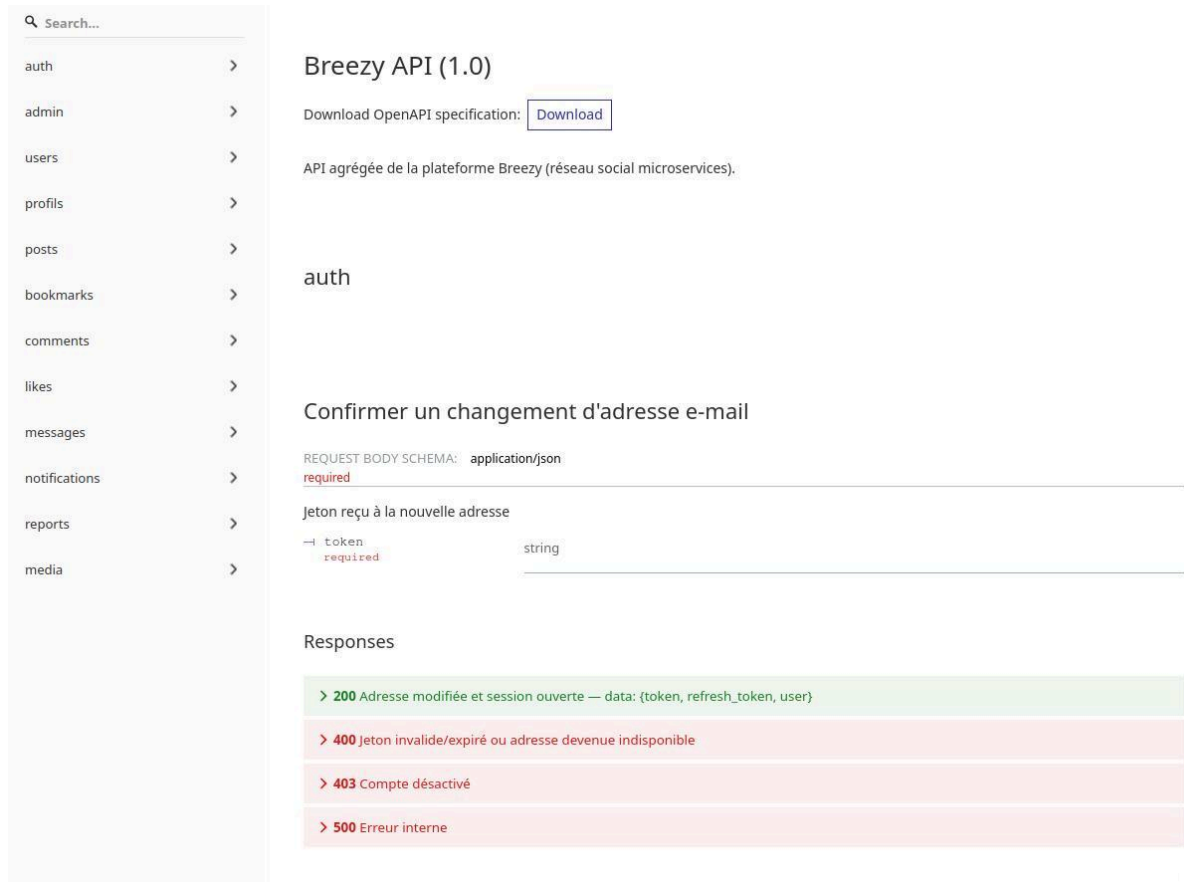
L'intégration est facilitée par Docker Compose. Les bases PostgreSQL, MongoDB et MinIO sont démarrées avec des volumes persistants, puis les services Go et le frontend sont lancés dans le même réseau Docker. Les fichiers .env.example et le Makefile rendent le démarrage plus standardisé. Les commandes make env, make dev, make up, make down et make reset structurent les usages courants. L'application a par ailleurs été déployée en production sur un VPS, avec déploiement automatisé des images.

Aspect	Outil	Rôle
Tests back-end	go test, go vet	Validation de middlewares, règles métier, services, repositories et fonctions de sécurité.
Tests frontend	Vitest, TypeScript, ESLint	Tests de fonctions UI, i18n, couleurs, mentions, hashtags, crypto et contrôle de typage.
CI	GitHub Actions	Workflows Go, frontend, intégration Docker, validation Swagger et contrôle de dérive documentaire.
Documentation	OpenAPI/Swagger, Redoc	Spécification agrégée des routes et documentation consultable par service.
Logs	slog structuré	Request ID, niveau de log, service, corrélation utilisateur et messages cohérents entre services.
Déploiement	Docker Compose + VPS	Orchestration reproductible en local et mise en production sur serveur dédié.

Tableau 14 : Outils de tests, qualité et déploiement

Ces outils ne remplacent pas une campagne de tests complète, mais ils donnent une base solide. Nous avons également réalisé des validations manuelles end-to-end : inscription, vérification e-mail, connexion, création de profil, publication, like, commentaire, follow, message privé, notification, upload média, changement de paramètres et accès modération/admin.

La documentation [OpenAPI/Swagger](#) joue un rôle de contrat entre le frontend et les services. Elle facilite la vérification des routes disponibles, des méthodes HTTP, des paramètres et des codes de retour. Les logs structurés aident à suivre les requêtes lors de l'intégration, notamment quand une action traverse la gateway puis plusieurs services.



Search...

- auth >
- admin >
- users >
- profils >
- posts >
- bookmarks >
- comments >
- likes >
- messages >
- notifications >
- reports >
- media >

Breezy API (1.0)

Download OpenAPI specification: [Download](#)

API agrégée de la plateforme Breezy (réseau social microservices).

auth

Confirmer un changement d'adresse e-mail

REQUEST BODY SCHEMA: `application/json`
required

Jeton reçu à la nouvelle adresse

token **required** string

Responses

- > **200** Adresse modifiée et session ouverte — data: {token, refresh_token, user}
- > **400** Jeton invalide/expiré ou adresse devenue indisponible
- > **403** Compte désactivé
- > **500** Erreur interne

Figure 8 : Page swagger

10. Sécurité, audits et tests d'intrusion

La sécurité a été traitée à la fois comme une exigence explicite du cahier des charges (sessions JWT, routes protégées) et comme un sujet d'ingénierie à part entière, jusqu'à la réalisation d'un test d'intrusion (pentest) sur l'application déployée.

10.1 Sécurité dès la conception

L'architecture applique le principe de défense en profondeur et ne fait jamais confiance au frontend :

- Validation du jeton dans chaque service. Le JWT est signé en HS256 (algorithme épinglé, alg:none rejeté) et vérifié signature et expiration par chaque service, et non uniquement par la gateway. L'identité provient toujours du jeton validé (et jamais du corps de requête ou de l'URL), ce qui protège contre l'usurpation d'objets d'autrui (IDOR/BOLA) et l'affectation de champs non autorisés.
- Sessions. Access token court (5 min) envoyé en Bearer, d'où une immunité CSRF native ; refresh token (24 h) dans un cookie httpOnly + SameSite + Secure, à rotation (révoqué à chaque usage). Un changement de mot de passe, une réinitialisation, un changement d'e-mail ou un bannissement révoque toutes les sessions.
- Isolation réseau. En production, seul le reverse-proxy (TLS) publie des ports ; les bases de données et les services restent sur un réseau interne, non joignables depuis l'extérieur.
- Hiérarchie des privilèges. Un modérateur ne peut pas agir sur un autre modérateur ou administrateur.

10.2 Protection des comptes et des données

Mécanisme	Mise en œuvre
Mots de passe	Hachés avec bcrypt (jamais stockés en clair).
Jetons de session et de compte	Hachés en SHA-256 côté serveur ; le secret en clair n'est jamais conservé.
Vérification d'e-mail	Obligatoire ; la création de contenu (publier, commenter, messenger) exige un e-mail vérifié.
Double authentification (2FA)	TOTP (applications d'authentification standard) ; secret chiffré AES-256-GCM ; anti-force brute (challenge invalidé après 5 codes erronés + limitation de débit).
OAuth Google & Github	Jeton d'identité vérifié (signature, émetteur, audience) ; liaison de compte uniquement si l'e-mail est vérifié, sans prise de contrôle via un second fournisseur.
Limitation de débit	Sur connexion, 2FA, inscription, mot de passe oublié, renvoi d'e-mail et traduction.
En-têtes HTTP de sécurité	HSTS, anti-clickjacking, anti-sniffing, Referrer-Policy, Permissions-Policy, politique de contenu (CSP).
Messagerie privée	Chiffrement de bout en bout (X25519 + XChaCha20-Poly1305), serveur aveugle au contenu, sauvegarde de clé protégée par passphrase (Argon2).
Téléversement de fichiers	Validation par signature binaire (et non par le type déclaré), liste blanche sans SVG, plafonds de taille, identifiants aléatoires.

Tableau 15 : Mécanismes de protection des comptes et des données

10.3 Test d'intrusion (pentest)

Un test d'intrusion a été conduit sur l'application en production. Il combine un audit boîte noire externe (12/06/2026) et un audit boîte blanche revue intégrale du code des onze services, de la gateway, du frontend et de l'infrastructure (18/06/2026). Le bilan global : l'application est solide sur son cœur (contrôle d'accès, sessions, injections, XSS, téléversement, secrets au repos, isolation réseau) ; les faiblesses restantes concernaient surtout le durcissement du périmètre d'authentification et l'hygiène de configuration.

Sévérité	Nombre	Suite donnée
Critique	0	
Élevée	0	
Moyenne	5	Majoritairement corrigées le jour même (rate-limiting, vérification e-mail, annuaire restreint, en-têtes de sécurité, anti-force brute 2FA).
Faible	7	Corrigées ou acceptées par décision produit (ex. fil public imposé par le client).

Tableau 16 : Synthèse des résultats du test d'intrusion

Points testés et non vulnérables: injections SQL (requêtes paramétrées) et NoSQL (filtres typés), usurpation d'objets d'autrui (propriétaire dérivé du jeton), affectation de champs non autorisés, XSS (échappement systématique, auto-lien restreint à http/https), SSRF (serveurs distants fixes), traversée de répertoires (stockage objet à clés opaques), exposition réseau (bases non publiées) et takeover OAuth.

Limites assumées : l'access token court réside côté navigateur (risque atténué par sa durée de vie de 5 minutes et l'immunité CSRF) ; l'analyse automatisée des secrets, le passage de la CSP en mode bloquant et un bannissement temporaire d'IP (type Fail2Ban) restent des chantiers de durcissement continu. Le pentest ayant été réalisé le 18/06/2026, il portait sur

le périmètre de cette date ; report-service, finalisé ensuite, suit les mêmes patrons de sécurité que les autres services et fera l'objet d'une passe d'audit dédiée.

9. Difficultés rencontrées et limites

La principale difficulté a été la coordination entre services. Dans un monolithe, l'affichage d'un profil avec ses posts pourrait se traiter dans une seule base. Dans Breezy, username, profil, posts, médias et relations appartiennent à des services différents. Il a donc fallu définir des contrats clairs et accepter que certaines vues soient composées côté frontend ou BFF.

La deuxième difficulté a concerné la sécurité des sessions. Le choix access token + refresh token améliore le contrôle des sessions, mais l'access token en localStorage reste exposé à une faille XSS. La durée courte du jeton limite l'impact, mais une application de production devrait renforcer la protection côté navigateur et généraliser des en-têtes de sécurité stricts.

La messagerie chiffrée a aussi demandé beaucoup d'attention. Elle combine chiffrement côté client, stockage de métadonnées, WebSocket, reçus et pièces jointes chiffrées. Ce module fonctionne comme démonstrateur avancé, mais il nécessiterait encore des audits cryptographiques et des tests plus poussés avant un usage réel.

Zone	Limite connue
Gateway	Le middleware JWT valide et propage les claims, mais certains préfixes doivent encore être protégés de manière plus systématique au niveau gateway.
Frontend	L'édition complète des posts côté interface reste à finaliser, même si les interactions principales sont déjà présentes.
Tests	Les tests unitaires sont nombreux, mais les tests d'intégration avec PostgreSQL, MongoDB et MinIO réels doivent être renforcés.
Sécurité	Le rate-limiting, la défense XSS et l'analyse de secrets doivent être améliorés avant exposition publique prolongée.
Recherche	La recherche actuelle couvre hashtags, usernames et noms affichés, mais elle reste moins puissante qu'un moteur full-text dédié.

Observabilité	Les logs sont structurés, mais il manque des métriques CPU/mémoire, traces distribuées et tableaux de bord complets.
---------------	--

Tableau 17 : Limites connues du projet

Ces limites n'empêchent pas le fonctionnement global : elles distinguent une application de projet d'une application prête pour une forte charge et une exposition publique prolongée. Les identifier explicitement fait partie de la démarche d'ingénierie attendue.

10. Axes d'amélioration et évolutions possibles

12.1 Axes d'améliorations

Les perspectives ci-dessous prolongent le travail sans changer l'architecture. Elles sont priorisées par le rapport effort / impact et visent surtout la robustesse, la sécurité et l'expérience utilisateur. L'effort est estimé en jours-homme (j/h) indicatifs.

Axe	Évolution proposée	Impact	Effort (j/h)	Priorité
Sécurité gateway	Protéger systématiquement les préfixes (politique public/protégé/admin).	Élevé	5	Haute
XSS et sessions	Réduire l'exposition de l'access token, CSP stricte, audit des entrées.	Élevé	8	Haute
Rate-limiting	Limites sur login, reset, publication, commentaires, messages, upload.	Élevé	4	Haute
Tests d'intégration	Bases réelles en CI pour les flux auth, post, message, notification, média.	Moyen	10	Haute
Modération	Étendre report-service : pré-modération assistée (détection automatique), file priorisée et journal d'audit des décisions.	Moyen	8	Moyenne
Recherche	Recherche full-text avancée avec index spécialisés ou moteur dédié.	Moyen	12	Moyenne
Performance	Optimiser les comptages followers/following, cache d'enrichissements.	Moyen	6	Moyenne

Médias	meilleure compression, miniatures, nettoyage des blobs orphelins.	Faible	7	Basse
Observabilité	Métriques CPU/mémoire, dashboards, alertes, traces distribuées.	Moyen	9	Moyenne
Industrialisation	Dependabot, gitleaks, registry d'images, déploiement continu complet.	Moyen	6	Moyenne

Tableau 18 : Axes d'amélioration priorités (effort/impact)

Ces évolutions sont réalistes et cohérentes avec l'état actuel du projet. Elles ne remettent pas en cause l'architecture microservices ; elles la renforcent progressivement pour la rapprocher d'une application exploitable sur une durée plus longue. Le chantier le plus structurant n'est toutefois pas technique mais économique : il s'agit de mettre en place le modèle de financement décrit ci-dessous.

12.2 Modèle économique et perspectives de financement

Statut : prospectif. La régie publicitaire n'est pas encore développée ; ce volet décrit la trajectoire de financement envisagée et en démontre la viabilité. Les montants sont des hypothèses illustratives et prudentes.

L'enjeu le plus structurant pour la pérennité de Breezy est la mise en place de son modèle économique. Le produit est techniquement prêt mais ne génère pas encore de revenus. La trajectoire envisagée suit la logique classique des réseaux sociaux, en trois temps :

1. Amorçage par levée de fonds. Dans ses débuts, Breezy ne vit pas de la publicité mais d'un financement par l'investissement (levée de fonds, capital d'amorçage). Ces fonds couvrent l'hébergement, le développement et surtout l'acquisition des premiers utilisateurs. À ce stade, l'objectif prioritaire est l'audience, pas le revenu.
2. Croissance de l'audience. Une base d'utilisateurs actifs suffisante est la condition pour intéresser des annonceurs : sans audience, pas de valeur publicitaire. L'effort porte sur la rétention, la viralité (invitations, partages) et la qualité de l'expérience.

- Monétisation publicitaire. L'audience constituée, Breezy active sa régie : posts sponsorisés (publicité native dans le fil) facturés au CPM (≈ 8 € pour mille impressions) ou au CPC ($\approx 0,25$ € par clic), tendances sponsorisées, et formules annonceurs en libre-service (Starter 99 €/mois, Pro 490 €/mois, Business 1 900 €/mois). Un abonnement Breezy+ (4,99 €/mois, sans publicité, badge, fonctions étendues) diversifie les revenus.

Cette logique utilisateurs d'abord, publicité ensuite justifie le recours initial à l'investissement : la publicité ne devient une source de revenus crédible qu'au-delà d'un certain volume d'audience. Le ciblage s'appuierait uniquement sur des données de première main (interactions sur la plateforme), dans une approche respectueuse de la vie privée et conforme au RGPD.

Sur des hypothèses prudentes coût de R&D du produit réalisé valorisé à $\approx 28\,000$ €, revenu publicitaire moyen par utilisateur croissant avec le ciblage, conversion progressive vers Breezy+ la projection à trois ans est la suivante :

Compte de résultat simplifié	An 1	An 2	An 3
Utilisateurs actifs mensuels (MAU)	50 000	200 000	600 000
Total revenus (publicité + Breezy+)	185 000 €	1 100 000 €	4 378 000 €
Coûts (infrastructure + équipe + acquisition)	168 000 €	520 000 €	1 850 000 €
Résultat annuel	+17 000 €	+580 000 €	+2 528 000 €
Résultat cumulé (R&D incluse)	-11 000 €	+569 000 €	+3 097 000 €

Tableau 19 : Projection financière à trois ans

Le résultat cumulé qui intègre l'investissement initial devient positif au cours de la deuxième année (seuil de rentabilité $\approx 14^e$ mois). Le modèle reste rentable en troisième année même

en dégradant de 30 % les hypothèses de remplissage publicitaire et de prix, ce qui confirme sa robustesse. La mise en place de cette régie publicitaire constitue donc le principal chantier produit une fois la levée de fonds réalisée et l'audience amorcée.

11. Conclusion

Breezy nous a permis de concevoir et de réaliser une application web distribuée autour d'un cas concret : un réseau social léger inspiré de Twitter/X. Le projet combine un frontend moderne Next.js/React/TypeScript, une API Gateway, plusieurs microservices Go/Gin et trois solutions de stockage adaptées aux responsabilités métier : PostgreSQL, MongoDB et MinIO. Les trois rôles attendus sont fonctionnels, l'ensemble est conteneurisé et déployé, et la couverture de tests dépasse 95 % sur les composants clés.

Le projet a aussi permis de mettre en pratique des sujets importants : authentification sécurisée, JWT et refresh token, double authentification (TOTP), vérification d'e-mail, messagerie chiffrée de bout en bout, séparation des données, communication REST, WebSocket, stockage média, internationalisation, modération et documentation d'API. La démarche a été poussée jusqu'à un test d'intrusion qui n'a révélé aucune faille critique ni majeure. Le passage de Node.js/Express vers Go/Gin n'a pas été présenté comme une simple préférence, mais comme une adaptation cohérente avec l'objectif de services indépendants et conteneurisables. La conduite du projet méthodologie agile, suivi GitHub Projects, planning en six phases, gestion explicite des risques et des retards a permis de livrer un produit cohérent dans des délais courts.

Au-delà des fonctionnalités visibles, le principal apprentissage concerne la séparation des responsabilités : chaque service doit posséder un domaine clair, une donnée claire et des contrats d'API compréhensibles. Le modèle économique fondé sur la publicité, complété par l'abonnement Breezy+, montre par ailleurs que le produit peut être financé de manière soutenable. Les limites identifiées rappellent qu'il reste du travail pour atteindre un niveau de production, mais Breezy constitue une base solide, réaliste et économiquement crédible pour un livrable étudiant de développement web full-stack distribué.

12. Table des figures

Tableau 1 : Indicateurs clés du projet.....	6
Tableau 2 : Objectifs, attentes du projet et traduction en livrables.....	11
Tableau 3 : Niveaux de priorisation des tâches.....	13
Tableau 4 : Couverture fonctionnelle (Fx1 - Fx23).....	15
Tableau 5 : Rôle de chaque service.....	19
Tableau 6 : Principes et flux d'architecture.....	21
Tableau 7 : Stack technique et justifications.....	23
Tableau 8 : Etapes de réalisation.....	26
Tableau 9 : Outils de pilotage et de collaboration.....	27
Tableau 10 : Planning par phases et jalons.....	28
Tableau 11 : Répartition des rôles de l'équipe.....	29
Tableau 12 : Contributions transverses par domaine.....	29
Tableau 13 : Gestion des risques et des retards.....	30
Tableau 14 : Matrice des permissions par rôle.....	43
Tableau 14 : Outils de tests, qualité et déploiement.....	65
Tableau 15 : Mécanismes de protection des comptes et des données.....	68
Tableau 16 : Synthèse des résultats du test d'intrusion.....	69
Tableau 17 : Limites connues du projet.....	72
Tableau 18 : Axes d'amélioration priorités (effort/impact).....	74
Tableau 19 : Projection financière à trois ans.....	76
Figure 1 : Roadmap du projet Breezy en six phases.....	5
Figure 2 : Architecture cible de Breezy en quatre couches : frontend, API Gateway et neuf services métier.....	17
Figure 3 : Propriété des données : « une donnée = un service ».....	20
Figure 4 : Flux d'authentification à double jeton JWT.....	33
Figure 5 : Flux de messagerie chiffrée de bout en bout (E2EE).....	38
Figure 6 : Flux de signalement et d'auto-masquage.....	41
Figure 7 : Coverage des services (Codecov).....	64
Figure 8 : Page swagger.....	66
Maquette 1 - Écran d'authentification.....	44
Interface 1 - Écran d'authentification.....	45
Maquette 2 - Fil d'actualité desktop.....	46
Interface 2 - Fil d'actualité desktop.....	47

Maquette 3 - Fil d'actualité mobile.....	48
Interface 3 - Fil d'actualité mobile.....	49
Maquette 4 - Page profil.....	51
Interface 4 - Page profil.....	52
Maquette 5 - Modale de publication.....	53
Interface 5 - Modale de publication.....	54
Maquette 6 - Messagerie.....	55
Interface 6 - Messagerie.....	56
Maquette 7 - Paramètres.....	57
Interface 7 - Paramètres.....	58
Interface 7 bis - Paramètres.....	59
Maquette 8 - Modération et administration.....	60
Interface 8 - Modération et administration.....	61
Interface 8 bis - Modération et administration.....	62